
Part II

Fundamental Principles

Astrophysical Recipes (Second Edition)

The art of AMUSE

Simon Portegies Zwart, Steve McMillan and Steven Rieder

Chapter 4

Gravitational Dynamics

Oh my God! — it's full of stars!

—Arthur C. Clarke

Here, we discuss how to solve relatively simple problems in Newtonian gravity using the monophysics solvers in AMUSE. Aside from the specific applications, this chapter gives an overview of how to perform simple experiments, what to be aware of, and how to address technical and algorithmic issues. This chapter should therefore be read as a general overview of the working of AMUSE with gravity as a focus. Even if you are not interested in gravity—for example, if your research is dominated by problems in radiative transfer or hydrodynamics—this chapter should still be your starting point.

4.1 In a Nutshell

4.1.1 Equations of Motion for a Self-Gravitating System

In the *Philosophiæ Naturalis Principia Mathematica*, Newton (1687) described his famous laws of motion in his chapter *Axiomata Sive Leges Motus* (the Axioms or Laws of Motion). He also laid the foundation for the general law of gravitation. In Newtonian gravity, the gravitational acceleration $\mathbf{a}_i$ of an object (subscript i) due to a group of objects (subscript j) is

$$\mathbf{a}_i \equiv \ddot{\mathbf{r}}_i = G \sum_{j \neq i}^N m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{|\mathbf{r}_j - \mathbf{r}_i|^3}. \quad (4.1)$$

Here, G is the gravitational constant, and m_j and $\mathbf{r}_j$ are the mass and position of particle j . Equation (4.1) is a set of coupled, second-order, nonlinear, singular ordinary differential equations. Its solutions exhibit a bewildering variety of

complex, often chaotic motions, and many books have been written about how to numerically solve it (Aarseth 1985, 2003; Heggie & Hut 2003; Binney & Tremaine 2008). Furthermore, because gravity is a long-range force, this behavior can be seen on virtually all scales, from planets and moons to clusters of galaxies. Newton's equations of motion break down only for very large scales or very strong gravitational fields. They are, however, applicable on most scales considered in this book.

4.1.2 Gravitational Timescales

Timescales are critical for understanding fundamental processes in (astro)physics. Therefore, we start each chapter on a new physical domain in AMUSE with a brief overview of the relevant timescales. In the case of self-gravitating systems, these are the *dynamical timescale*, the *relaxation timescale*, the *dynamical friction timescale*, and (in extreme circumstances) the *gravitational radiation timescale*.

4.1.2.1 The Dynamical Timescale

The dynamical timescale is the scale associated with orbital motion in a gravitating system. Also known as the crossing time, it is generally considered as the time taken for a typical particle to cross the system,

$$t_{\text{dyn}} = R/v, \quad (4.2)$$

where R is a characteristic dimension and v is a typical particle speed. In practice, R is often taken to be the mean radius of the system (defined more carefully below) and v is the root mean square velocity $\langle v^2 \rangle^{1/2}$ of all particles relative to the center of mass.

For a two-body gravitating system, the dynamical timescale is simply the orbital timescale. More generally, the dynamical timescale is the timescale on which a self-gravitating system establishes virial (or dynamical) equilibrium, in which there is no net flux of particles across any radius (see the assignment on virial equilibrium in Section 4.7.5).

For a system in virial equilibrium, the virial theorem (Collins 1978) relates the (time-averaged) kinetic energy T and potential energy U of the system by

$$2T + U = 0. \quad (4.3)$$

For a system of mass M , we can write $T = \frac{1}{2}M\langle v^2 \rangle$ and $U = -\frac{1}{2}GM^2/R_{\text{vir}}$, where the latter expression defines the virial radius R_{vir} . In that case, the virial theorem implies

$$\langle v^2 \rangle = \frac{GM}{2R_{\text{vir}}}. \quad (4.4)$$

Taking $R = R_{\text{vir}}$ (and neglecting constants of order unity), we can then write the dynamical timescale as

$$t_{\text{dyn}} = \sqrt{\frac{R_{\text{vir}}^3}{GM}}, \quad (4.5)$$

which we recognize as a generalization of Kepler's third law.

In defining the dynamical timescale, the virial radius is often replaced with the easier-to-measure half-mass radius, R_h , which is the radius of the sphere enclosing half of the total system mass. Although they are commonly used interchangeably, these two radii are distinct physical quantities. However, Spitzer (1987) notes that $R_{\text{vir}} \simeq 0.8R_h$ for a broad range of common dynamical models in virial equilibrium.

For an open star cluster with $R = 1$ pc and $M = 10^3 M_\odot$, or a globular cluster with $R = 10$ pc and $M = 10^6 M_\odot$, we find $t_{\text{dyn}} \simeq 0.5$ Myr. For our Galaxy, with $R = 10$ kpc and $M = 10^{11} M_\odot$, $t_{\text{dyn}} \simeq 50$ Myr. Within AMUSE, we can easily perform such a calculation as follows:

```
import numpy as np
from amuse.units import constants

def dynamical_time_scale(mass, radius, G=constants.G):
    return np.sqrt(radius**3 / (G * mass))
```

Having imported the needed libraries and defined this function, you might try the following:

```
from amuse.units import units
mass = 10**11 | units.MSun
radius = 10 | units.kpc
time_dyn = dynamical_time_scale(mass, radius)
print(f'dynamical time scale={time_dyn}')
```

The first two lines initialize the parameters M and R . The function is then called to calculate the dynamical timescale, and the result is printed as

```
dynamical time scale=12.2404614048 1.21539420228e+14 * s
```

You will probably find that the time is printed in a rather strange set of units. This reflects the way in which AMUSE carries out its dimensional calculations, operating on quantities and their units separately until the result is needed. It is usually best to convert into a defined set of units with

```
print(f'dynamical time scale={time_dyn.in_(units.Myr)}')
```

this produces more readable output

```
dynamical time scale=47.1433302078 Myr
```

You will see many more examples like the above code example in this book. We encourage you to copy and paste these code snippets into a Python terminal for execution and experimentation, but note that leading spaces as well as single quotes may not be copied correctly from the PDF version of this book into your Python environment. Alternatively, to cut and paste from the book, you can find all the

source code in the AMUSE repository in the module: `amuse.examples` (see also Section 1.4.3).

4.1.2.2 The Relaxation Timescale

The timescale on which the global characteristics (bulk system parameters and stellar orbital elements) of the system change is the relaxation timescale, t_r . In stellar systems, relaxation is driven by two-body encounters among stars, which change stellar orbital parameters and provide a means for energy and angular momentum to diffuse around the system, causing long-term structural evolution.

The two-body relaxation time is the timescale on which a “typical” particle has its orbit changed by (mostly distant) encounters with other particles. The relaxation time for a system of identical particles of mass m , number density n , and (three-dimensional) velocity dispersion $\langle v^2 \rangle$ is (Spitzer 1987)

$$t_r \simeq \frac{0.065 \langle v^2 \rangle^{3/2}}{nm^2 G^2 \ln \Lambda}, \quad (4.6)$$

where Λ is the ratio of the size of the system to the “strong encounter distance” $p_0 = Gm/\langle v^2 \rangle^{1/2}$, the impact parameter that would result in a 90° deflection for two typical cluster stars. For an idealized system of N identical objects in virial equilibrium, Spitzer (1987) finds $\Lambda = \gamma N$, with $\gamma = 0.4$. For more realistic systems, γ is probably considerably smaller: the value $\gamma = 0.11$ is in common use (Giersz & Heggie 1994; Heggie & Hut 2003).

A convenient global measure of relaxation in a system is the half-mass relaxation time, t_{rh} , which is obtained by replacing all terms in Equation (4.6) by their global averages and assuming virial equilibrium:

$$t_{rh} \simeq 0.138 \frac{N}{\ln \gamma N} t_{\text{dyn}}. \quad (4.7)$$

In AMUSE, we could expand the earlier code fragment from Section 4.1.2.1 to

```
import numpy as np
from amuse.units import constants, units

def dynamical_time_scale(mass, radius, G=constants.G):
    return np.sqrt(radius**3 / (G * mass))

def relaxation_time_scale(number_of_particles, mass, radius, G=constants.G):
    return (
        0.138 * number_of_particles / np.log(0.4 * number_of_particles)
        * dynamical_time_scale(mass, radius, G)
    )

number_of_particles = 500000
mass = 10**11 | units.MSun
radius = 10 | units.kpc
time_relax = relaxation_time_scale(number_of_particles, mass, radius)
print(f'relaxation time scale={time_relax.in_(units.Gyr)}')
```

For the Milky Way, with $\text{number_of_particles} \sim 10^{11}$ and t_{dyn} as derived above, this gives $t_{\text{rh}} \simeq 2.7 \times 10^7$ Gyr, which indicates that relaxation is unimportant. In the jargon of the field, such systems are termed *collisionless*. However, for an open star cluster, with $N \sim 10^3$, $t_{\text{rh}} \simeq 11$ Myr and we can expect considerable dynamical evolution in much less than the age of the Galaxy or the lifetime of the cluster. This is an example of a *collisional* dynamical system (note that the term has nothing to do with physical stellar collisions, which we will discuss in subsequent chapters).

4.1.2.3 The Dynamical Friction Timescale

Chandrasekhar (1943) showed that the gravity of a massive body moving through a uniform background distribution of low-mass objects creates a “wake”—an over-dense region—behind itself [see also Fellhauer & Lin (2007)]. The gravitational pull of this wake always acts oppositely to the massive body’s motion, leading to dynamical friction that tends to slow it down. If the low-mass background has density ρ and is (reasonably) characterized by an isotropic velocity distribution $\phi(v)$ (where $\int 4\pi v^2 \phi(v) dv = 1$), then the acceleration of the massive body due to dynamical friction against the background is

$$\mathbf{a}_{\text{df}} = -\frac{16\pi^2 G^2 M \rho \ln \Lambda \mathbf{V}}{V^3} \int_0^V v^2 \phi(v) dv, \quad (4.8)$$

where M and $\mathbf{V}$ are the mass and velocity, respectively, of the massive body, and Λ is defined as before—except that we replace p_0 with the radius R of the massive body if $R > p_0$. Note that, unlike relaxation, this expression does not depend explicitly on either the number density or the mass of the low-mass background, just on the total density.

If (again reasonably) the velocity distribution of the background is thermal,¹ such that

$$\phi(v) = \frac{1}{(2\pi\sigma)^{3/2}} e^{-v^2/\sigma^2}, \quad (4.9)$$

where σ is the one-dimensional root mean square velocity of the low-mass particles, this expression becomes (assuming $M \gg m$)

$$\mathbf{a}_{\text{df}} = -\frac{4\pi^2 G^2 M \rho \ln \Lambda \mathbf{V}}{V^3} \left[\text{erf}(x) - \frac{2x}{\sqrt{\pi}} e^{-x^2} \right], \quad (4.10)$$

where $x = v/\sqrt{2}\sigma$ (Binney & Tremaine 2008). The term in square brackets is zero for $x = 0$, scales as x^3 for small x , and tends to 1 for large x , effectively equaling 1 for $x > 3$.

¹ More precisely, this generally means that the direction is isotropic and the speed is drawn from a Maxwell–Boltzmann velocity distribution (Maxwell 1860) associated with some temperature.

The dynamical friction timescale is

$$t_{\text{df}} = \frac{V}{a_{\text{df}}} = \frac{0.059 V^3}{G^2 M \rho \ln \Lambda}, \quad (4.11)$$

where we have conventionally evaluated the bracketed expression in Equation (4.11) for $x = 1$. Combining Equations (4.6) and (4.11) (and neglecting differences in the definition of Λ), we find

$$t_{\text{df}} = 0.91 \left(\frac{V^2}{\langle v^2 \rangle} \right)^{3/2} \left(\frac{m}{M} \right) t_{\text{r}}. \quad (4.12)$$

Thus, so long as V^2 is comparable to $\langle v^2 \rangle$, a massive body will sink toward the center of a galaxy or cluster on a timescale much shorter than the relaxation time of the background particles.

4.1.2.4 The Gravitational Radiation Timescale

Relativistic effects, caused (for example) by two nearby point masses on a close orbit, have a characteristic timescale that is usually much longer than any of the other dynamical timescales. We therefore tend to ignore them in N -body simulations, but it is still good to be aware of their magnitudes.² The timescale for orbital precession (often referred to as apsidal precession) is shortest, but it has limited effect on the global dynamical evolution of a stellar system, except in close resonant or von Zeipel–Lidov–Kozai resonance interactions (Naoz 2016). The relativistic apsidal precession (in radians per orbital period) for a two-body orbit of semimajor axis a , eccentricity e , and period P , is

$$\epsilon = 24\pi^3 \frac{a^2}{P^2 c^2 (1 - e^2)}. \quad (4.13)$$

Here, c is the speed of light. For the planet Mercury, this turns out to be about 5×10^{-7} radians per orbital period of $P \simeq 88$ days. It takes about a century for Mercury to make an extra loop around the Sun due to this effect.

The timescale for inspiral due to the emission of gravitational radiation is considerably longer, but more relevant for cluster dynamics. We can estimate this timescale using a post-Newtonian (PN) expansion of the orbital dynamics (Peters 1964). Ignoring several complications, we arrive at

$$t_{\text{gwr}} \simeq 0.02 \frac{c^5 a^4 (1 - e^2)^{7/2}}{G^3 M m (M + m)}, \quad (4.14)$$

where M and m are the masses of the orbiting bodies, and a and e are the Newtonian orbital semimajor axis and eccentricity, respectively. Adopting representative values ($M \simeq 1.44 M_{\odot}$, $m \simeq 1.39 M_{\odot}$, $a \simeq 2.8 R_{\odot}$, and $e \simeq 0.62$) for the famous Hulse–Taylor pulsar (Hulse & Taylor 1975), we find $t_{\text{gwr}} \simeq 300$ Myr.

² Several integrators listed in Table 4.1 includes post-Newtonian corrections up to order 2.5.

The merger of such objects will result in a burst of gravitational waves. The first detection of such waves was made by the LIGO consortium on 2015 September 14 (Abbott et al. 2016).

4.1.2.5 Other Timescales

Many other timescales can be identified and defined. For example, timescales for specific processes, such as resonant relaxation (Rauch & Tremaine 1996) or the von Zeipel–Lidov–Kozai (von Zeipel 1910; Lidov 1962; Kozai 1962)³ cycles in hierarchical triple systems, may be relevant in certain circumstances. In Sections 8.4.5 and 9.4.3, we present examples for which these additional timescales are important.

4.1.3 Star Cluster Dynamics

Stars form in clustered environments, and much of the story of stars in our Galaxy has to do with the dynamical evolution of star clusters. The N -body codes described in this chapter are central to this study. Because this topic plays such a key role in our narrative, we pause here to introduce some terminology, a few common diagnostics having to do with basic cluster dynamics, and an evolutionary timeline. We will return frequently to these concepts in our later discussion. The dynamics we describe are undoubtedly going on in, and may be modified by, the presence of a surrounding gas cloud, but the essential underlying processes are largely unchanged.

We, regrettably, can only scratch the surface of this interesting field of research, so we refer the reader to the extensive literature for more in-depth discussion (see, e.g., Spitzer 1987; Heggie & Hut 2003; Binney & Tremaine 2008).

4.1.3.1 Cluster Structure and Diagnostics

We begin with a newborn group of stars formed from an interstellar cloud (see Chapter 6). Both theory and observation (e.g., Gnedin et al. 2015) suggest that these stars are unlikely to have formed in precise virial (dynamical) equilibrium (Section 4.1.2.1). However, simulations indicate that the system will virialize and form a more or less spatially smooth cluster in virial equilibrium within a few dynamical times, erasing any substructure associated with the formation process. For typical cluster parameters, the timescale to form a single, well-defined cluster is a few million years. From this point on (at least, to the extent that we can neglect overall rotation), we are justified in treating the cluster as a roughly spherically symmetric object. Consequently, simulations of the evolution of clusters after the formation stage generally use idealized Plummer- (Plummer 1911) or King-like (King 1966) models (Section 3.2) as initial conditions.

By this stage, clusters are simple enough that they can be described by a few parameters. The virial (or, as we have seen, the half-mass) radius defines the overall cluster scale. Theoretical cluster models generically predict—and most observations show—the presence of a roughly constant-density central region known as the core.

³The Lidov paper was originally published in 1961 in Russian as *Iskusstvennye Sputniki Zemli*, #8, p. 5; the cited article is the English translation.

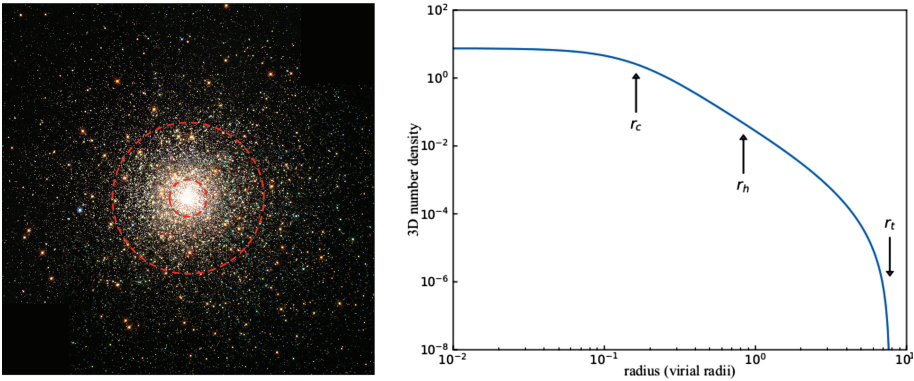


Figure 4.1. (a) The globular cluster M80 lies 10 kpc away and contains several hundred thousand stars. The cluster’s core and half-mass radii are indicated by dashed red circles. The field of view is 3 arc minutes across. Original image credit: The Hubble Heritage Team (AURA/STScI/NASA/ESA). (b) The King model that best fits the cluster, with core, half-mass, and tidal radii marked.

Observers define the *core radius*, r_c , as the distance from the cluster center at which the surface brightness drops to half the central value. Because density is generally hard to measure in N -body models, theorists use a somewhat different definition, defining the core radius as the density-squared weighted root mean square stellar distance from the cluster center. (We also think that this sounds complicated!) Perhaps remarkably, the two definitions seem to broadly agree. The ratio of the core radius to the half-mass radius (or to the cluster’s tidal radius in the ambient Galactic tidal field) is a key measure of cluster structure.

The cluster’s *Lagrangian radii* are, by definition, the distances from the cluster center containing specified fractions of the total cluster mass. The 50% Lagrangian radius is the half-mass radius. Studying selected Lagrangian radii of key mass groups provides critical insight into the cluster’s evolving density distribution. Figure 4.1(a) shows a Hubble Space Telescope (HST) image of the globular cluster M80. The cluster’s core radius and half-mass radius are indicated by dashed red circles. Figure 4.1(b) shows the density profile of the best-fitting King model (with $W_0 = 7.5$; see Section 3.2). The cluster’s tidal radius lies well outside the image shown in part (a); for many clusters, the tidal radius is determined from the best-fitting King model.

4.1.3.2 Essential Dynamics

As we just saw, dynamical friction causes the most massive stars in the cluster to sink toward the center (Spitzer 1969). The timescale can be very short, often less than the evolutionary lifetimes of the stars themselves (see Section 4.1.2.3). The most important consequences of this process are: (1) the surrounding lower-mass stars gain energy and expand outward as the massive stars lose energy and sink to the center of the cluster potential well; (2) the high density of the massive stars near the center means that the rate of both dynamical interactions (including binary formation) and physical collisions between stars (see Sections 8.5.2 and 8.5.4) is greatly increased, which influences the dynamical evolution of the system; (3) when stars lose mass—for

example, due to winds or supernovae—the effect on the cluster is much stronger when the mass loss comes from the center rather than the periphery.

Mass segregation is often interpreted in terms of energy equipartition, the expected end-point of relaxation, which should lead to $\langle mv^2 \rangle = \text{constant}$ for every stellar mass group. As a result, more massive stars slow down and sink toward the center of the cluster. However, perhaps surprisingly, the degree to which a system actually achieves equipartition is still not fully understood. Trenti & van der Marel (2013) reported that energy equipartition is never actually reached in HST observations and N -body simulations of clusters with realistic mass functions. On the other hand, simulations of more idealized two-component systems (containing just “light” and “heavy” stars; see below) indicate that equipartition is achieved, at least when the component mass ratio is not too large. The fine details of this fundamental process remain to be resolved.

On longer timescales, the escape of high-velocity stars (evaporation) and the internal effects of two-body relaxation, which transports energy from the inner to the outer regions of the cluster, lead to a phenomenon known as core collapse (Antonov 1962; Cohn 1980). During this phase, the central portions of the cluster accelerate toward infinite density while the outer regions expand. This process can be understood by recognizing that, according to the virial theorem [Equation (4.3)], a self-gravitating system has negative specific heat, so reducing its energy causes it to heat up. Hence, as energy moves from the (dynamically) warmer central core to the cooler outer regions, the core contracts and gets warmer, accelerating the contraction. The timescale for the process to reach completion (i.e., a core of formally zero size and infinite density) is $t_{\text{cc}} \sim 15t_{\text{rh}}$ for a system of identical masses (Hénon 1961), and less in the case of a broad spectrum of masses (Inagaki & Saslaw 1985), for which the core-collapse timescale falls to $t_{\text{cc}} \sim 0.2t_{\text{rh}}$ (Portegies Zwart & McMillan 2002).

Figure 4.2 illustrates some of these dynamical stages. It shows the evolution of a simple two-component stellar system, in which both components are initially distributed as the same $W_0 = 5$ King model. The mass ratio of the two components (1 and 2) is $\mu = m_2/m_1 = 3$, and the ratio of total masses is $M_2/M_1 = 1.6 \times 10^{-2}$. According to Equation (4.7), the half-mass relaxation time is $t_{\text{rh}} \simeq 300$ time units. Mass segregation, dynamical equipartition, core collapse, and dynamical binary formation and evolution in the more massive component can all be seen.

Several physical processes can counteract core collapse. As just mentioned, stellar mass loss (due to stellar winds or supernovae) tends to unbind the cluster, and mass segregation amplifies the effect on the core. In addition, binary systems, whether present initially in the cluster or formed dynamically (see Section 4.7.3), also play an important role. Briefly, “soft” binaries that have less binding energy than the mean stellar kinetic energy of the system tend to be destroyed by encounters with other stars. However, “hard” binaries (with binding energy greater than the mean) tend to become more tightly bound following an encounter, effectively heating the stellar system (Hills 1975; Heggie 1975). We will discuss in more detail how binary interactions and dynamics are handled in AMUSE in Section 8.7.1.

One important guideline in understanding global cluster dynamics is Hénon’s principle (Hénon 1975): When two coupled dynamical processes operate on very

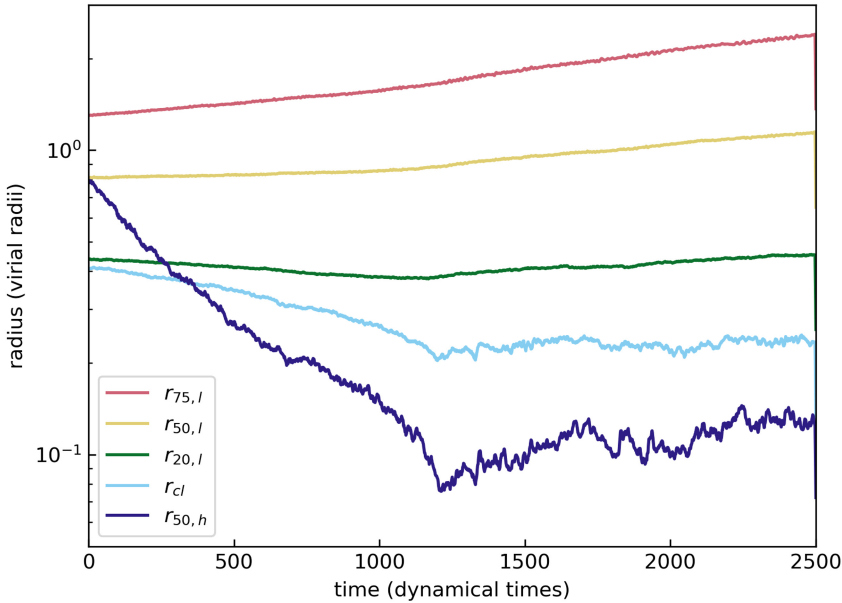


Figure 4.2. Evolution of a simple two-component system comprising 16k “light” (1) and 88 “heavy” (2) stars, with mass ratio $m_2/m_1 = 3$. The half-mass radius of the massive component, the core, and 20%, 50%, and 75% Lagrangian radii of the light component, are shown. The data have been smoothed using a square window of total width six time units. Mass segregation occurs during the first 600 time units; subsequently, both the massive system and the light core contract in quasiequilibrium. The system remains close to energy equipartition for $t > 600$. Eventually, after core collapse at ~ 1200 time units, binary formation and dynamical heating cause both the massive system and the light core to expand. (Data courtesy of M. Brewer.)

different timescales—here, core energy production and the much slower outward conduction of energy through the half-mass radius—the faster process will always come into equilibrium with the demands of the slower one. As a result, the overall evolution of the cluster—long-term expansion and dissolution in the Galactic tidal field—is largely independent of the detailed physics operating in the core. Regardless of the precise core dynamics, a typical cluster is expected to dissolve in the Galactic tidal field in about 10 half-mass relaxation times (Lee & Ostriker 1987).

There is a considerable literature about the late stages of star cluster evolution, in what is often referred to as the postcollapse phase, during which so-called gravothermal oscillations may play a major role (Bettwieser & Sugimoto 1984; Makino 1996). Although this epoch is of considerable interest from a theoretical perspective, it is not clear what its observational relevance is in our relatively young universe.

4.1.4 Physics of the Integrator

Evaluating Equation (4.1) is not hard, nor is integrating the equations of motion, but it is important to appreciate the limitations of the adopted methods. It is sometimes said that gravity solvers, and N -body solvers in particular, have no underlying assumptions, and that all physical processes are properly taken into account. This, of course, is

untrue. They solve the equations of motion in the nonrelativistic, point-mass approximation, quietly neglecting the spatial extent of the gravitating objects and the effects of their tidal interactions. For many studies, this does not pose a problem, but it is nevertheless important for the user to be aware of these shortcomings.

In solving the fundamental equations that govern the dynamical evolution of a self-gravitating system, we generally assume that the following conditions are met.

1. **Point-mass approximation.** An important and common underlying assumption is that all particles are point masses. Even if the particles should have nonzero sizes, their nonsphericity is generally ignored. The center of mass is assumed to be the geometric center of the particle. The consequences of this assumption are probably very small for most applications, but there certainly are cases where it is unwarranted. In Section 4.5.1, we will discuss a case in which the size of objects actually drives part of the physics, and it will turn out that dimensions of stars and planets are important drivers for interesting physical phenomena. In many hydrodynamical applications (discussed in Chapter 6), the finite size of objects motivates the performance of numerical simulations. Note that in Section A.5.2 we will explain the more complex aspects of interrupting one code to hand the control to another.
2. **Newton is correct.** The force in Equation (4.1) is pure Newtonian dynamics, ignoring relativistic corrections. The improvements made by Einstein (1916) and Verlinde (2017) are usually ignored. Some implementations of the N -body problem in AMUSE include additional terms that take relativistic effects into account, but most do not. Currently, there is no treatment of modified Newtonian dynamics (Milgrom 1983) in AMUSE, and no prescription for dark energy. These purported physical effects may be taken into account by modifying the spatial geometry or by introducing corrections to the force law, realized via the Bridge coupling strategy (see Chapter 9). Dark matter can be mimicked by introducing additional gravitating particles, although this is a coarse-grained approximation to the real physics.
3. **Gravity acts instantaneously.** In classical theories of gravitation, the “speed of gravity” is the rate at which a change in the distribution of energy or momentum of matter results in a change in the gravitational field it produces. In a more physically correct sense, the “speed of gravity” refers to the speed of a gravitational wave. In Newtonian N -body dynamics, this speed is infinite; in Einstein’s theory of general relativity, it is the speed of light. This discrepancy can be safely neglected in many simulations because typical gas and stellar velocities are much smaller than the speed of light, except in the close proximity of massive black holes.

4.1.4.1 Numerical Accuracy

The digital computers on which we perform our calculations are often limited to only 16 decimal places.⁴ This sounds like a lot for everyday calculations, but it is

⁴In 64 bit IEEE-754 compliant numerical representation, each floating-point number is presented by 1 bit for the sign, 11 for the exponent, and 52 for the mantissa.

quite limited if one considers that some simulations may require $\gtrsim 10^{16}$ operations (10 Petaflops), in which case round-off must play a potentially important role. This amount of computing corresponds roughly to the number of operations required to integrate the orbit of the Moon over the lifetime of the solar system. The effect of round-off is rarely studied in detail in N -body calculations, and the assumption is always made that it does not affect the long-term evolution of the system.

The solutions of an N -body problem are deterministic. Initial conditions define one and only one trajectory in phase space. Each initial point in phase space then maps to a unique final state of the system. In practice, we expect the calculated phase-space trajectory of an N -body simulation to differ from the true phase-space trajectory— N -body simulations give the wrong results (see Portegies Zwart & Boekholt 2018). Deviations from the trajectory, and the eventual location of the final state, can be caused by external perturbations, numerical round-off, and other sources of error. These sources of error may themselves be nondeterministic (e.g., in case partial forces are calculated in parallel). In Section 4.4.3, we present an example of how to measure the degree of chaos in an N -body system, by comparing a more accurate solution with a reprehensible solution.⁵

If computer errors are truly random and without systematic effects, the final state of an ensemble of inaccurate calculations should be centered around the true final state. Even though we do not know the shape or the dispersion of this distribution, the mean should still be consistent with the true solution. In a computer simulation, any initial point in phase space will map to a final state-point, but this is not the true final point. It is commonly believed that if we generate an ensemble of initial points in phase space and calculate them all to their final points in phase space, the distribution of final points will represent the true solution for the given initial distribution.

Every implementation of the N -body problem has its own characteristic numerical fingerprint that can be recognized by the growth of the numerical error. This is caused, in part, by specific choices of additional parameters, such as the time-stepping scheme or the introduction of a softening length that limits the depth of the potential. These choices are often hidden deep within the code, and it is widely assumed that they do not materially affect the outcome of the simulation. Despite these potential sources of error and uncertainty, our confidence in N -body simulations rests on the basic observed fact that all implementations lead to similar outcomes when applied to standard problems.

The detailed long-term evolution of a star cluster—virialization, mass segregation, core collapse, and long-term evolution (see Section 4.1.3)—appears to be substantially independent of both the initial model and the specific integration technique used. This has been repeatedly verified in a host of code comparisons over the past three decades (Chernoff & Weinberg 1990; Giersz & Heggie 1994; Fukushige & Heggie 1995; Giersz & Heggie 1996; Takahashi & Portegies Zwart 1998; Spinnato et al. 2003; Anders et al. 2012; Whitehead et al. 2013; Rodriguez et al. 2016). As a rule of thumb, we can trust predictions of the bulk or statistical properties of the system, but we should not take

⁵ Solution to Newton's equations of motion for which the accumulation of numerical errors and the system's response exceeds the exponential growth of the initial offset δ (Portegies Zwart & Boekholt 2018).

individual stellar orbits too seriously, although there are clear counterexamples where the chaotic equations of motion are truly solved. In AMUSE, such arbitrarily precise calculations to a converged solution can be performed using the Brutus direct N -body code.

4.2 N -body Integration Strategies

In principle, the force calculation and the pushing of the particles are separate operations in a gravitational dynamics code, but they are coupled in most “pure” N -body implementations (see Section 4.2.2). Calculating the force in Equation (4.1) is straightforward, but the number of operations for the force on a single particle scales as the number of particles [$\mathcal{O}(N)$], and calculating the forces on all particles then scales as $\mathcal{O}(N^2)$, which becomes impractical for current computers without graphics processing unit (GPU) acceleration for $N \gtrsim 10^4$.

4.2.1 Global Structure of an N -body Code

Figure 4.3 presents a simple workflow for a basic N -body code. A handy programmer can probably rewrite this in about 100 lines of source code. Such codes can be (and have been) tuned for parallel operation, GPUs, and other high-performance/exotic hardware, and they form an excellent basis for numerical studies. Converting the flow chart in Figure 4.3 into a working program is not entirely trivial, but we refrain from going into more detail here because there are many excellent textbooks that do just that (e.g., Aarseth 2003; Heggie & Hut 2003).⁶

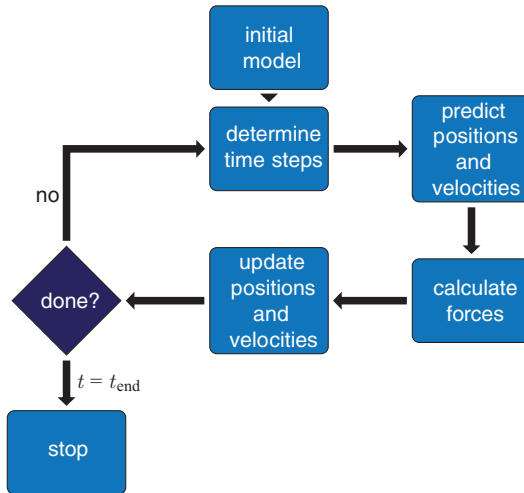


Figure 4.3. Workflow for a typical direct N -body code.

⁶ For an illustration of how this problem can be addressed in a number of programming languages, see <https://github.com/spzward/NBabel>.

The heart of an N -body code is the integration scheme, the mathematical time-discretization used to solve Equation (4.1). An important consideration in choosing an integration scheme is its order, which tells us how the algorithmic error (per step) δE scales with time step δt . For a scheme of order k , $\delta E \propto \delta t^k$. Typical schemes in common use are of second-, fourth-, or sixth-order. Higher-order schemes are preferred in direct-summation N -body calculations because a given accuracy can be achieved using longer time steps (remember here that the number of force operations per particle per step is $\mathcal{O}(N)$).

The program illustrated in Figure 4.3 uses a predictor–corrector scheme to advance the particles in time (see Section 4.2.3). There are many alternative approaches, but these schemes have become the de facto standard in astrophysical N -body simulations.

4.2.2 Types of N -body Code

Several families of algorithms have been developed to solve the N -body problem. Some scale as $\mathcal{O}(N)$ (Ambrosiano et al. 1988; Dehnen 2014a), some scale better, $\mathcal{O}(N \log N)$ (Barnes & Hut 1986), but some scale much worse, $\mathcal{O}(N^3)$ (Boekholt & Portegies Zwart 2015). The differences are rooted in optimization, where computational speed is, broadly speaking, traded for accuracy. We draw a distinction between *pure* N -body, *direct* N -body, and *approximate* N -body codes.

- **Pure N -body** codes solve Newton’s equations of motion with no free physical parameters, although most have some capacity to flag special events, such as close encounters or binary dynamics. The only adjustable quantity in a pure N -body code is the time-stepping criterion used to integrate the equations of motion. Examples are `ph4` and `Hermite`, both based on the fourth-order Hermite predictor–corrector scheme (Makino & Aarseth 1992) (cf. Hut et al. 1995 for a lower-order approach). Tunable precision N -body codes are an interesting subset of pure N -body codes. They go beyond the standard double precision of digital computers, and allow energy conservation to be tuned to arbitrary precision. The code `Brutus` (Boekholt & Portegies Zwart 2015) has an extra free parameter, which controls the mantissa of the floating-point operations.
- **Direct N -body** codes solve Newton’s equations of motion with additional parameters, often to speed up the code or to reduce round-off. Many such codes incorporate regularization techniques for resolving close encounters and close multiple subsystems, and/or neighbor schemes to reduce the force calculation time. The cost per step of these codes still scales as the square of the number of particles. `NBODY6` is a prime example (Aarseth 1985; Harfst et al. 2007).
- **Approximate methods** relax the strict N^2 procedure for computing the gravitational force. These include various flavors of tree codes, particle-mesh, and other multipole treatments. Such methods are very popular, mainly because they are very fast. The most common example is the Barnes–Hut tree scheme (Barnes & Hut 1986), in which forces from nearby particles are calculated directly, but more and more distant particles are grouped together into clumps

of increasing size, whose gravity is approximated by the first few terms of a multipole expansion. Very often, for computational efficiency, only the monopole term is retained—when calculating the orbit of the Sun in our Galaxy, it is reasonable to treat the Andromeda galaxy as a point mass. The result is a reduction of the $\mathcal{O}(N^2)$ complexity of a direct force calculation to $\mathcal{O}(N \log N)$. Examples of tree codes in AMUSE include `BHTree` (Barnes & Hut 1986), `MI6` (Iwasawa et al. 2015), and `Bonsai` (Bédorf et al. 2012; Bédorf et al. 2014). Another popular method is the fast multipole method (Greengard & Rokhlin 1987; Ambrosiano et al. 1988), or the equally fast but considerably more precise code `gyrfalcon` (Dehnen 2014a, 2014b), which scales even more favorably— $\mathcal{O}(N)$.

- **Hybrid methods** Hybrid N -body codes try to combine the advantages of high precision with the speed of tree codes. They often scale favorably compared to direct integration, but at a slightly reduced precision. One example, available in AMUSE is `Petar` (Wang et al. 2016). There are several scripts in AMUSE that exhibit similar advantages, such as the `Nemesis` (van Elteren et al. 2019) module and `LonelyPlanets` (Cai et al. 2019). In `LonelyPlanets`, we reduce these costs through a two-stage divide-and-conquer strategy (Cai et al. 2019; Stock et al. 2020). In stage one we simulate a dynamical evolution and stellar evolution of a cluster in the Galactic potential, in stage two we integrate the planetary systems with only the nearest perturbing cluster members. This strategy easily saves a factor of 10^4 in computer time when integrating a cluster of 1000 stars, of which 30 receive a system of 10 planets each. The cost of such a two-stage process is reduced by ignoring the effect individual planets have on the stars, but one loses the ability to follow the orbits of rogue planets in the cluster.

4.2.3 Discretization Strategies in N -body Simulations

A system of differential equations, such as Equation (4.1), can be discretized in time and solved using a variety of numerical techniques. Two basic strategies for obtaining a solution are *explicit* and *implicit* schemes. An explicit method provides a formula or procedure that allows the state of the system at time $t + \delta t$ to be determined entirely from the state of the system at time t . In an implicit method, the state of the system at time $t + \delta t$ also appears in the formula, leading to an implicit equation that generally must be solved iteratively.

Implicit methods tend to be more stable and (in principle) allow much longer time steps, although they are generally more expensive due to the iterative solution entailed. However, the longer time steps are often unattainable due to the chaotic and singular nature of the underlying equations—the characteristic timescales of the motion can differ by many orders of magnitude from particle to particle and from one time to another. As a result, explicit methods are generally favored over implicit methods for N -body applications.

Many explicit schemes exist, but by far the most commonly used approach in N -body simulations of stellar clusters is the predictor–corrector scheme (see Section 4.2.2). Historically, these methods appeared as the first iteration in implicit solutions

to the discretized equations of motion, but in their modern incarnation they have become explicit solvers.

Broadly speaking, predictor–corrector schemes proceed in three steps:

- The predictor step uses whatever derivative information is available at time t (usually at least the acceleration, but often higher derivatives as well) to extrapolate all positions and velocities at time $t + \delta t$ as Taylor series (Taylor 1715).
- The accelerations (and higher derivatives) are then reevaluated using the predicted positions and velocities, and the new and old accelerations are combined to yield estimates of higher derivatives. For example, we can estimate the “jerk” (the derivative of the acceleration) as $j \simeq (a^{\text{new}} - a^{\text{old}})/\delta t$.
- The corrector step then refines the predicted positions and velocities using the additional derivative information, essentially adding extra terms to the Taylor series.

These schemes are compact, easy to program, and generally proceed with a single evaluation of the accelerations at each step (the new accelerations are reused in the next prediction), making them very efficient in terms of computer resources.

The second-order predictor–corrector scheme (often called the velocity-Verlet method, and sometimes implemented as the “Leapfrog” scheme; see Verlet 1967) is very widely used; we discussed in Chapter 1 how this integrator finds its origin in the Babylonian civilization between 350 and 50 BCE. Note that the terminology between Leapfrog, Verlet, and Babylonian schemes are a bit confusing. The velocity-Verlet does not correct the new positions. The predictor and corrector steps are usually reserved for methods that do at least one iteration, but possibly more. Although of low order, its simplicity and the fact that it is symplectic (see Section 4.2.3.2) make it an attractive choice in many applications. The fourth-order Hermite scheme (Makino 1991) is a natural generalization of the Verlet method (except that it is not symplectic). It uses accelerations and jerks, and has for some time been the method of choice in large N -body simulations. The last decade has seen the development and implementation of sixth- and eighth-order versions in large-scale integrators. Again, it goes beyond the scope of this book to dwell on this topic given that excellent texts already exist. We direct the interested reader to Nitadori & Makino (2008).

One disadvantage of explicit schemes is that errors on successive steps tend to be correlated, so the cumulative error grows faster than the random walk expected for uncorrelated errors (Brouwer 1937), for which the net error would grow as the square root of the number of steps. This can be solved with compensated Kahan–Babuška summation (Kahan 1965; Babuška 1969). In most applications, this more rapid error growth is an acceptable trade-off for the improved speed of the method.

The recently revived family of hybrid methods, which combine the good qualities of two or more integration algorithms (e.g., McMillan & Aarseth 1993), have very promising characteristics. Recent examples include *Pentacle* (Iwasawa et al. 2017) and *Petar* (Wang et al. 2016), which combine a Hermite direct code with a Barnes-Hut tree code. The implementation in *Petar* adds

algorithmic regularization and PN effects to allow massive compact objects in a stellar cluster to be addressed.

4.2.3.1 Particle–Particle–Particle–Tree Schemes and Other Advanced Schemes

The development of a hybrid N -body integrator with the good energy conserving quality of the high-order explicit predictor–corrector scheme and the excellent performance of the hierarchical tree code approach is with the family of PP–PT (or P^{3T} , sometimes referred to as *Pikachu*) schemes (Iwasawa et al. 2015).

The PP–PT scheme combines direct N -body integration (particle–particle) for close interactions with the Barnes–Hut tree algorithm (particle–tree) for long-range interactions. The code *Petar* implements this scheme. *Petar* is designed for simulations of globular clusters, including stellar evolution, binary treatment, and PN effects. The algorithmic regularization (Hellström & Mikkola 2010) allows accurate treatment of close encounters and binary systems. *Petar* is particularly well suited for simulating dense stellar systems, such as globular clusters, galactic nuclei, and young star clusters. Its hybrid approach and use of modern computational techniques make it a powerful tool for studying complex astrophysical phenomena involving gravitational dynamics.

Finally, it is worth mentioning some recent Kepler-splitting methods, in which the problem is separated into N Kepler problems plus perturbations. The superposition principle is subsequently applied to knit the N^2 Kepler solutions together into a coherent N -body solution. The methods work, parallelize almost trivially, and the errors do not grow on long timescales (compared to the dynamical time), making such methods very suitable for studying the long-term dynamical evolution of the solar system (Wisdom & Holman 1991; Yoshida 1993; Duncan et al. 1998; Chambers 1999; Laskar & Robutel 2001; Saha & Tremaine 1992; Rein & Liu 2012; Liu et al. 2016).

4.2.3.2 Symplectic and Time-Reversible Schemes

In the context of long-time-scale calculations, the family of symplectic integrators are of particular interest. The defining property of a symplectic scheme is that it preserves phase-space area in a dynamical system. It follows that the phase-space volume element is preserved—the familiar Liouville theorem. Given that Hamiltonian systems also have these properties, symplectic integration of Hamiltonian systems yields numerical solutions that stay faithful to the true solutions, in the sense that physically conserved quantities are conserved numerically over long periods of time. Such schemes are also time-reversible, meaning that the initial conditions can be recovered (to machine accuracy) by running a simulation backward in time (see also Tremaine 2015). Note here that time reversibility should not be confused with a converged solution, which requires robust brute force calculations with *Brutus*.

In a symplectic scheme, it is common to split the underlying Hamiltonian into slowly and rapidly varying parts. For example, when discussing the orbits of minor bodies in the solar system, we could rewrite the Hamiltonian of the multibody system as

$$H = H_{\text{kep}} + H_{\text{int}}. \quad (4.15)$$

Here, we follow the example of Wisdom & Holman (1991). The leading term H_{kep} represents interactions between the minor bodies with the Sun, while H_{int} represents interactions among the minor bodies. (The latter term could, in turn, be further separated into changes on timescales that are short or long compared to the dynamical time.) Each component of the split Hamiltonian can be solved independently, using explicit schemes, and the solutions combined. We will return to symplectic schemes in Chapter 9, where we will adopt this approach as our standard for combining codes.

Explicit symplectic schemes can be very accurate, approaching machine precision with sufficiently small time steps, even when the equations are stiff. Their main drawback is that they require the time steps for all objects to be the same. Such shared time step schemes are often employed for planetary, galactic, and cosmological simulations, but are prohibitively expensive for star cluster simulations, which generally require individual and time-variable steps (Aarseth 1985). We further discuss the Babylonian or Verlet scheme in Section 9.1.1.

4.2.3.3 General Relativistic Effects

Most N -body codes in AMUSE adopt Newton's equations of motion, but there are a few exceptions that also solve for the general relativistic equations of motion. This is generally realized through a Taylor expansion of the Einstein–Infeld–Hoffman equations of motion (Einstein et al. 1938), called the PN expansions (Poisson & Will 2014). They elegantly balance accuracy with computational efficiency, allowing researchers to probe the relativistic dance of celestial bodies without the full complexity of Einstein's field equations.

The expansion unfolds as a series in powers of v/c , where v is the system's typical velocity and c is light speed. This approach modifies the spacetime metric and equations of motion, introducing relativistic corrections to Newtonian dynamics. As the expansion progresses through higher orders, it captures increasingly subtle effects, such as the perihelion precession (first-order Taylor expansion, or commonly called the 1-PN term) and gravitational waves. Many pseudorelativistic N -body codes incorporate the following terms:

1-PN: Terms up to $(v/c)^2$ leads to perihelion precession.

2-PN: Terms up to $(v/c)^4$ leads to gravitational time dilation.

2.5-PN: Includes radiation reaction effects.

3-PN and higher: More accurate but computationally intensive.

These expansions tend to break down when the orbital velocity approaches the speed of light, or in the close proximity to an event horizon. It is, however excellently suited for simulating orbits of black hole binaries in stellar clusters. The spacetime metric is expanded around the flat Minkowski metric, with corrections added at each order. The geodesic equation of motions are expanded, resulting in modified Newtonian equations of motion with relativistic corrections.

The underlying assumption for generating the PN expansions is that all black holes operate in pairs. Although a reasonable assumption so long as black holes are binary members, it breaks down for democratic triple (or higher-order multiple) black holes.

4.2.3.4 Parallelization and Hardware Acceleration

The N -body problem is relatively easy to parallelize. Several algorithms are known to perform well for each of the families of integration strategies mentioned in Section 4.2.2. However, we will not dwell on the details because many excellent papers and textbooks have already been written on these topics. The problem is also well suited to hardware acceleration GPUs. Although worthy competitors exist, the most common acceleration hardware currently used in the gravitational N -body community is the NVIDIA GPU, programmed using the CUDA language.⁷ These tiny computers combine large amounts of computing power with relatively low prices, consistently leading the market in both teraflops per dollar and teraflops per watt. Many of the N -body codes listed in Table 4.1 are parallel, GPU-accelerated, or both.

4.3 The AMUSE Interface to Gravity Solvers

The preceding discussion of gravity solvers may mask a very basic fact: the codes all have very similar input and output. This is necessarily so, since they aim to solve the same physics. We take advantage of this in AMUSE to define a minimal Gravity Interface that can be used to access any gravitational solver in a homogeneous way.

A simple script to integrate Newton's equations of motion is presented in Code example 4.1. You can test it yourself with the following code:

```
import amuse.examples as ae
ae.run("gravity_minimal")
```

We will dissect the complete example in some detail here. The script starts by importing the required AMUSE packages:

```
from amuse.units import nbody_system
from amuse.ic.plummer import new_plummer_model
from amuse.community.hermite import Hermite
```

The first import, of `nbody_system`, makes a *unit system* available (see Section 2.3). The second import statement loads a function `new_plummer_model` that generates the initial conditions (see Chapter 3). Finally, the third import loads a class, `Hermite`, which is an example of a community code that implements the gravitational dynamics interface. All the codes that conform to this interface (we will encounter more of them later) implement in some way (as discussed in Section 4.2.2) a solver for Equation (4.1).

The code in Code example 4.1 is as simple as we could make it—and honestly, it does not do much. It can, however, be the basis for more rewarding investigations and it does contain in essential form all necessary ingredients of more complicated scripts. As we will see in later chapters, it can be expanded and used for far more

⁷ See <https://developer.nvidia.com/cuda-zone>

```

1  """
2  Minimal example for gravity calculations in AMUSE
3  """
4
5  from amuse.units import nbody_system
6  from amuse.ic.plummer import new_plummer_model
7  from amuse.community.hermite import Hermite
8
9
10 def gravity_minimal(number_of_stars=100, time_end=1 | nbody_system.time):
11     """
12     Initialises a Plummer sphere of length 'number_of_stars' and integrates until
13     'time_end' with the Hermite integrator
14     """
15     stars = new_plummer_model(number_of_stars)
16
17     gravity = Hermite()
18     gravity.particles.add_particles(stars)
19
20     energy_total_initial = gravity.kinetic_energy + gravity.potential_energy
21
22     gravity.evolve_model(time_end)
23
24     energy_total = gravity.kinetic_energy + gravity.potential_energy
25     energy_error = (energy_total_initial - energy_total) / energy_total_initial
26
27     print(
28         f"model time = {gravity.model_time} "
29         f"total mass = {stars.total_mass()} "
30         f"total energy = {energy_total} "
31         f"relative energy error = {energy_error}"
32     )
33
34     gravity.stop()
35
36
37 def main(**kwargs):
38     gravity_minimal(**kwargs)
39
40
41 if __name__ == "__main__":
42     main(number_of_stars=100, time_end=1 | nbody_system.time)

```

Code example 4.1: Minimal AMUSE code for solving a simple N -body problem. Source code is available in `amuse.examples.gravity_minimal`.

complex simulations. This minimal script, and many of the following scripts, is set up as follows:

- Generate a realization of the starting conditions (Section 4.3.1),
- Specify and initialize the (gravity) solver to use (Section 4.3.2),
- Evolve the model to a specific time (Section 4.3.4),
- Print diagnostics,
- Produce a figure of the results or analyze the data (Section 2.2.3),
- Shutdown the community code.

After loading in the appropriate packages, the Python interpreter processes the definition of the functions `main` and `gravity_minimal` and proceeds to the line

```
if __name__ == "__main__":
```

This piece of Python code tells the interpreter that the following lines are to be ignored when the script is not run standalone (see Appendix C). It is standard practice in Python to do this because it makes your code easily reusable: another script can import the function `gravity_minimal` without triggering execution of a run.

Here, the main function is called with its default parameters,

```
{main} (number_of_stars=100, time_end=1 | nbody_system.time)
```

Gravitational N -body codes tend to employ dimensionless units or Hénon units (Hénon 1972; Heggie & Mathieu 1986). We briefly notice that in this system, the gravitational constant G , the total cluster mass M , and the cluster virial radius R_{vir} (see Section 4.1.2.1) each have a value of 1. For a cluster in virial equilibrium, the kinetic, potential, and total energies then are (Hénon 1971; Heggie & Mathieu 1986)

$$T = \frac{1}{4}, \quad U = -\frac{1}{2}, \quad E = -\frac{1}{4}. \quad (4.16)$$

N -body units are handy when we are interested in the outcome of a pure gravitational problem. Internally, N -body integrators have almost universally adopted them, as well as in N -body initial condition generating functions such as `new_plummer_model`, `new_king_model`, etc. (see Section 3.2). They greatly facilitate comparison between different codes. In addition, they afford the minor computational advantage of keeping the magnitudes of most physical variables close to unity, although they make little difference—at best, avoiding a single multiplication by G in each force calculation. On the downside, dimensionless units have caused confusion among generations of students and other noncognoscenti in their attempts to reconcile physical and N -body quantities. And as soon as other physical processes intrude, such as when combining gravitational dynamics with stellar evolution or hydrodynamics, N -body units rapidly lose their usefulness.

In reality, a typical N -body code does not care at all about units. It accepts a number with no questions asked, except perhaps whether it is a floating-point number or an integer. However, the AMUSE interface does care about dimensional and nondimensional units because it mediates the conversion between the units used in different modules. For this reason, any calculation involving N -body units that plans to use other modules with units must be calibrated by defining a `converter` specifying the N -body mass, length, and/or timescales in physical units (see Section 2.3). In Section 4.4 (Code example 4.3 in particular), we introduce the command-line argument parser `argparse`, and present an example of a script in which we use this with units (see Section 2.4.2).

```

10 import matplotlib.pyplot as plt
11 from amuse.io import read_set_from_file
12
13
14 def plot_gravity(
15     filename="sun_venus_earth.amuse",
16     length_unit=units.au,
17 ):
18     particles = read_set_from_file(filename)
19
20     figure = plt.figure(figsize=(8, 8))
21     ax = figure.add_subplot(1, 1, 1)
22
23     colormap = ["yellow", "green", "blue"] # specific to a 3-body plot
24     size = [40, 20, 20]
25     edgecolor = ["orange", "green", "blue"]
26
27     for i, snap in enumerate(particles.history):
28         ax.scatter(
29             snap.x.value_in(length_unit),
30             snap.y.value_in(length_unit),
31             c=colormap,
32             s=size,
33             edgecolor=edgecolor,
34         )
35     ax.set_xlabel(f"x [{length_unit}]")
36     ax.set_ylabel(f"y [{length_unit}]")
37
38     save_file = "plot_gravity.png"
39     plt.savefig(save_file)
40     print(f"\nSaved figure in file {save_file}\n")

```

Code example 4.3: Minimal routine to plot N -body simulation data. Full source code is available in `amuse.examples.plot_gravity`.

4.3.1 Generating Initial Conditions

The main routine in Code example 4.1, where the real work is done, starts with

```

def gravity_minimal(number_of_stars=100, time_end=1 | nbody_system.time):

```

the declaration of the function that actually does the work. The next line (see also Section 3.2.1)

```

stars = new_plummer_model(number_of_stars)

```

generates the initial density and velocity of the particles, in this case a Plummer model (Plummer 1911) with “number_of_stars” stars. In Section 3.2.1 we present an example output for this routine.

4.3.2 Specifying and Initializing the Gravity Solver

The next two lines in the code example initialize an instantiation of the N -body integrator, in this case the `Hermite()` integrator, and send the previously created

generated particles to the integrator. Much of the magic in AMUSE is in this simple line:

```
gravity = Hermite()
```

The result is a detached process called `hermite_worker` that starts on your computer, running simultaneously with the Python program that spawned it. You can check that multiple processes are now running on your computer by typing

```
%> ps aux | grep hermite
```

on the command line (use `top -o cpu` on a Mac), which in our case shows something like (where we left out a few columns):

USER	PID	%CPU	%MEM	TTY	STAT	START	TIME	COMMAND
spz	342887	0.0	0.0	pts/3	S+	10:49	0:00	hermite_worker 55101 127.0.0.1
spz	342888	0.0	0.0	pts/3	S+	10:49	0:00	hermite_worker 55101 127.0.0.1
spz	343018	0.0	0.0	pts/10	S+	10:50	0:00	grep --color=auto hermite

The two `hermite_worker` processes are the N -body integrator and its interface, both running at the host IP address (127.0.0.1). In an earlier version of AMUSE, the code would continue consuming 100% CPU time due to polling the AMUSE framework asking for things to do. Considering the modern desire to spend fewer cycles, and less electricity, we improved on this behavior, and have silenced the polling. The Python instance `gravity` is our portal to the N -body code.

Because the N -body code is a detached process, we must send it a termination signal from the AMUSE script at the end of the calculation. Otherwise, the N -body code will keep running in the background, until the Python interpreter cleans up (“garbage collects”) the `gravity` variable. To stop `Hermite`, we terminate it by

```
gravity.stop()
```

This will send a kill signal to `Hermite`, and all data in the code will be lost.

In the example, `gravity` is an instance of a class implementing the AMUSE gravity interface. This object manages and transparently handles communication with the `hermite_worker` process or any other N -body code (see Section 2.2.3). Any of the other gravity solvers can be used in the same way by changing the call to the integrator. For example,

```
gravity = Bhtree()
```

instantiates and initializes the gravity solver `BHTree`, a Barnes–Hut tree code. Aside from this line, the interface is the same in all cases, as long as the other code is a gravity code.

4.3.3 Feeding Particles to the N -body Code

So far, we have not yet informed the N -body code which data it should use. These data come in the form of a particle set. In Section 4.3.1, we initialized a particle set that we called `stars`. We can send this particle set to the N -body code by adding the `stars` to the code already running

```
gravity.particles.add_particles(stars)
```

We now have two sets of particles—the Python particle-set `stars` and the set that now lives in the memory of the running code. The latter set can be addressed directly by querying `gravity.particles`. Once we tell the gravity solver to advance in time, these two particle sets will diverge. The particle set in Python will remain unchanged, but the copy resident in the integrator will be updated by the community code, here an instance of `Hermite`.

4.3.4 Evolving the Model to the Desired Time

The actual computational work is done in

```
gravity.evolve_model(time_end)
```

Here, the N -body code runs to time `time_end`, i.e. the solution to Equation (4.1) is advanced to the desired end time. After the code is done, we calculate the kinetic and potential energies.

To test the accuracy of the integrator, we print the final energies and the energy error since the system was initialized.

We are now ready to run the minimal script with the following arguments

```
%> python -m amuse.examples.gravity_minimal
```

which will produce the following (slightly edited to fit the text column) output:

```
T= 1.0 time M= 1.0 mass E= -0.250000001272 mass * length**2 * time**-2
Q= -0.5068237292374607 dE= -5.089542761314269e-09

In this session you have used the AMUSE modules below.
Please cite any relevant articles:

"AMUSE-Hermite (framework 2023.10.1.dev1+gb159086e4.d20240124)"
  ADS:1995ApJ...443L..93H (Hut, P., Makino, J. & McMillan, S., *
  Astrophysical Journal Letters* , **443**, L93-L96 (1995))

"AMUSE (unknown version)"
  DOI:10.5281/zenodo.1435860
  ADS:2018araa.book....P (Portegies Zwart, S. & McMillan, S.L.W., 2018)
  ADS:2013CoPhC.183..456P ** (Portegies Zwart, S. et al., 2013)
  ADS:2013A&A...557A..84P ** (Pelupessy, F. I. et al., 2013)
  ADS:2009NewA...14..369P (Portegies Zwart, S. et al., 2009)
```

This final message reports to the user the codes and components used and provides references that we hope they will cite if the code and script are run for production and/or publication purposes. If this message annoys you, it can be suppressed:

```
from amuse.support import literature
literature.TrackLiteratureReferences.suppress_output()
```

This concludes our first acquaintance with an AMUSE simulation script. Of course, some elements are still missing to be able to analyze and change our simulation. We want to be able to examine the data in more detail (Section 2.2.3), store the data for publication and analysis (Section 2.2.4), and change to the units used to communicate with the code (Section 2.3). Also, in order to change the numerics of the simulation, we need to be able to change solver parameters (Section 2.3.3) and to change to a different solver (Section 4.3.7).

4.3.5 Scaling the Simulation

So far, we have discussed only dimensionless N -body units. These are convenient for performing scale-free simulations, as are common in N -body simulations. In Section 2.3, we presented a more complete overview of how to declare and use converters in AMUSE. Here, we just reiterate the most critical parts needed to understand this chapter.

A `converter` is simply an object that contains knowledge of the units in a problem, which can be passed as needed as an argument to other functions requiring this information. For example, a unit converter can be provided when initiating an N -body solver:

```
gravity = Hermite(converter)
```

This establishes the connection between dimensionless and physical units. Now, we can feed particle sets to the N -body code in any units we choose, and retrieve dimensional information in any convenient set of units. The conversion is automatic as long as we stay within one system of units,

```
print(f"Earth's mass is: {(1 | units.MEarth).in_(units.kg)}")
```

or

```
print(
    f"The Sun's luminosity is about {(3.8e+33 | units.erg/units.s).in_(units.LSun)}")
)
```

4.3.6 Code Parameters

As we explained in Section 2.3.3, the working of various codes can be controlled and tuned through parameter settings. There we explained that we try to give these parameters the same name throughout a physical domain, but sometimes fail in this objective.

One case where we failed to be consistent is in the two direct N -body codes `ph4` and `Hermite`. The parameters to control the size of the time step in these codes are called `timestep_parameter` and `dt_param`, respectively. A comparable parameter in `BHTree` is called `time_step`. However, the latter is not inconsistent because `time_step` really sets the actual time step in `BHTree`. The differences in implementation are the main reason for failing to come up with a unique parameter name, which is a consequence of these codes being written by different authors with slightly different objectives. This forms a natural inconsistency in AMUSE.

A parameter where we succeeded in finding a common naming across various codes is the gravitational softening parameter ϵ (Aarseth 1963). This is a very important parameter, used in many N -body codes. Softening removes the singularity in the inverse-square force by replacing Equation (4.1) with

$$\mathbf{a}_i = G \sum_{j \neq i}^N m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{[(\mathbf{r}_j - \mathbf{r}_i)^2 + \epsilon^2]^{3/2}}. \quad (4.17)$$

This is done to accommodate integration schemes that cannot handle point-mass potentials. Column 8 of Table 4.1 below lists whether or not a code is (or can be) softened. In Section 8.7.1, we describe a module that adds point-mass support to any otherwise softened code. The square of the softening parameter is generally referred to as `epsilon_squared` in all N -body codes. For example, ϵ^2 can be set to the squared mean interparticle spacing, in N -body units, with

```
gravity.parameters.epsilon_squared = 1./N**(2./3)
```

In case you prefer to look at good old-fashioned source code, these parameter settings are all defined in the interface file of each community module in the AMUSE source code. For the `Hermite` package, for example, the parameters can be found in the module `amuse.community.hermite.interface`, in the function `define_parameters()`.

4.3.7 Changing the Solver Used in the Simulation

Each community code in AMUSE can be imported with:

```
from amuse.community.{codename} import {Codename}
```

Note here that the first letter of the code name is capitalized, followed by lowercase. For example, to import the `BHTree` and `ph4` codes:

Table 4.1. Overview of the N -body Codes Incorporated in AMUSE

Name	Ref	Type	Order	Language	N	Soft.	Parallel	GPU
Brutus	1	Pure	2nd	C++	$\lesssim 10$	No	Yes	No
Hermite		Pure	4th	C++	$\lesssim 10^4$	n/y	Yes	No
ph4		Pure	4th	C++	$\lesssim 10^5$	n/y	Yes	Yes
HiGPU's	4	Pure	6th	C++	$\lesssim 10^4$	n/y	Yes	Yes
phiGRAPE	5	Pure	4th	F77	$\lesssim 10^5$	n/y	Yes	Yes
SmallN		Pure	4th	C++	$\lesssim 10$	n/y	Yes	No
Rebound	7	Direct	Var	C	$\lesssim 10^4$	No	Yes	No
Huayno	8	Direct	Synpl.	C	$\lesssim 10^3$	n/y	Yes	Yes
Mercury	9	Direct	Synpl.	F77	$\lesssim 10^4$	No	No	No
Octgrav	10	Approx.	TC 2nd	C++	10^4 – 10^7	Yes	Yes	Yes
Bonsai	11	Approx.	TC 2nd	CUDA	10^4 – 10^{11}	Yes	Yes	Yes
MI6	12	Approx.	PN 4th $\rightarrow$ 6th	C++	$\lesssim 10^6$	n/y	Yes	Yes
Mikkola	13	Direct	PN 6th $\rightarrow$ 4th	F77	$\lesssim 100$	No	No	No
Twobody		Direct	Kepler	Python	2	No	No	No
Sakura	15	Direct	Kepler	C++	$\lesssim 10^5$	n/y	Yes	No
Nbody6++	16	Direct	4th	F77	$\lesssim 10^5$	No	No	Yes
BHTree	17	Approx.	TC 2nd	C++	10^3 – 10^6	Yes	Yes	No
SecularMultiple	18	Approx.	orbit av.	C++	$\lesssim 10$	No	No	No
Hermite_GRX	19	Pure	4th	C++	$\lesssim 10^3$	n/y	Yes	No

Here, TC refers to tree code (see Section 4.2.2), and PN to the possibility of including post-Newtonian terms. References: 1 Boekholt & Portegies Zwart (2015), 4 Spera et al. (2012); Capuzzo-Dolcetta et al. (2013), 5 Harfst et al. (2007), 7 Rein & Liu (2012, 2011), 8 J  nes et al. (2014), 9 Chambers & Migl  rini (1997), 10 Gaburov et al. (2010), 11 B  dorf et al. (2012), 12 Oshino et al. (2011); Iwasawa et al. (2015), 13 Mikkola (1983); Mikkola & Merritt (2008), 15 J  nes et al. (2014), 16 Aarseth (1985); Wang et al. (2015), 17 Barnes & Hut (1986), 18 Hamers & Portegies Zwart (2016), and 19 Portegies Zwart et al. (2022). The columns give the order of the integrator, the language used, the number of particles for which the code is most suitable and whether or not the code can handle softening, is parallelized, or is graphics processing unit enabled.

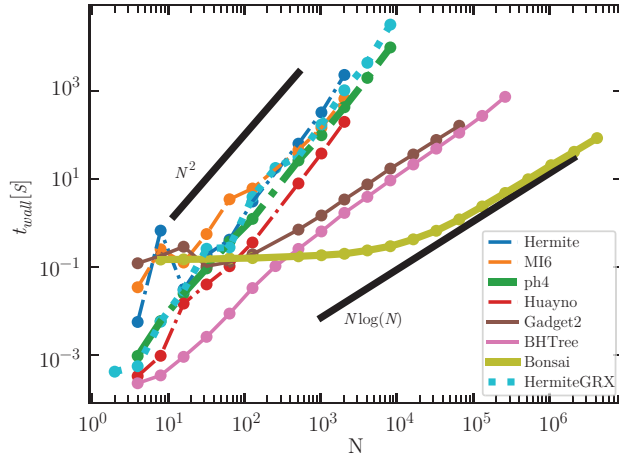


Figure 4.4. Wall-clock time as a function of N for a range of N -body solvers in AMUSE, integrating of a Plummer sphere for 1 N -body time unit. The runs were performed for 10 N -body time units. A script to generate this figure can be found at `amuse.examples.plot_nbody_performance`.

```
from amuse.community.bhtree import Bhtree
from amuse.community.ph4 import Ph4
```

Table 4.1 gives a brief overview of the current codes in AMUSE and their capabilities, and A.8 contains a complete (at the time of writing) list of the AMUSE modules. Note that while some codes, such as `ph4`, `SmallN`, `Huayno`, `BHTree`, and `Bonsai`, were added for strategic reasons, to fill a specific need, many others have been contributed by AMUSE users who used the framework to tackle some problem.

Figure 4.4 shows the performance of a variety of N -body solvers in AMUSE. Initial conditions are those of a Plummer sphere in virial equilibrium. No softening was used, but the runs were carried out for only 10 N -body time units; no close encounters occurred during this short time, and therefore no softening was necessary. We see two distinct performance behaviors in Figure 4.4. At the upper left, we have a group of direct N -body codes with $O(N^2)$ scaling. At right center, we see the $(N \log(N))$ behavior characteristic of tree codes. Code-to-code differences span about an order of magnitude and are the result of implementation details. The fastest code (bottom red curve) is the GPU-native `Bonsai`; the hardware characteristics of the GPU architecture explain its relatively poor performance for up to about 1000 particles but much better performance for larger numbers of particles.

4.4 Examples

4.4.1 Integrating the Orbits of Venus and Earth

In Section 3.4.1, we presented a plot of the integrated orbits of the Sun, Venus, and Earth (Figure 3.4). Now, we can improve our understanding of the scripts used to make this figure.

First, we generate the Cartesian coordinates for the Sun, Venus, and Earth, much as we did in Code example 3.7:

```
particles = new_solar_system()
SunVenusEarth = Particles()
SunVenusEarth.add_particle(particles[particles.name=='SUN'])
SunVenusEarth.add_particle(particles[particles.name=='VENUS'])
SunVenusEarth.add_particle(particles[particles.name=='EARTHMOO'])
SunVenusEarth.move_to_center()
print(SunVenusEarth)
```

Further down in Section 4.4.1.2, we show how you can make a realization of the solar system at a specific time.

Code example 3.6 presents the initialization routine, in which the masses, positions, and velocities of the Sun, Venus, and Earth are stored in a particle set. After initialization, the particle set is moved to the center of mass of the three-body system, to prevent the entire system from drifting, since we initialized it with the Sun at the origin. Subsequently, if we chose, we could give the solar system a position and systematic velocity in space by writing

```
particles.position += (-8.5, 0.0, 0.0) | units.kpc
particles.velocity += (11.1, -228.3, 7.25) | units.kms
```

These are the approximate position and velocity of the Sun relative to the Galactic center. These lines are best included after repositioning the solar system to its center of mass—otherwise the Sun, rather than the solar system’s barycenter, would be placed at this location (although the difference is very small).

The integration routine presented in Code example 4.2 follows the orbital motion of Venus and Earth. In the first few lines, we load specific parts of AMUSE. Given the particle set in the argument list, we use the total mass and the distance to one of the particles (Earth—1 au) to initialize the converter. This converter is subsequently used to initialize the N -body code. We then add the particles to the N -body code and make special references to the two planets, Venus and Earth.

In the following lines, we initialize lists of x and y positions, which we update in the subsequent loop over time, with a time step of one day. In this case, we do not bother to initialize a channel to communicate between the framework and the running code. The function with a channel would be essentially the same, but would run slightly faster. After the time loop, we stop the N -body code and return the x - and y -position lists of the two planets.

Instead of storing the positions of Venus and Earth in arrays, we could write snapshots to a file at the end of each loop, using `write_set_to_file()`, and subsequently analyze the data with a separate script. To improve our diagnostics, we might also print or store the energy error produced by the N -body integrator at every diagnostic output interval.

```

51 from amuse.units import nbody_system
52 from amuse.community.huayno import Huayno
53
54
55 def integrate_solar_system(
56     particles, time_step=1 | units.day, time_end=3 | units.yr,
57 ):
58     convert_nbody = nbody_system.nbody_to_si(
59         particles.mass.sum(),
60         particles[1].position.length(),
61     )
62
63     gravity = Huayno(convert_nbody)
64     particles_in_code = gravity.particles.add_particles(particles)
65
66     while gravity.model_time < time_end:
67         gravity.evolve_model(gravity.model_time + (time_step))
68         particles_in_code.new_channel_to(particles).copy()
69         particles.savepoint(timestamp=gravity.model_time)
70
71     gravity.stop()
72
73     return particles

```

Code example 4.2: Program to integrate the orbits of Venus and Earth around the Sun. The full code example is in `amuse.examples.sun_venus_earth`.

4.4.1.1 Plotting the Results

We can subsequently plot the resulting data file from the simulation using the `pyplot` routines in the `matplotlib` package. An example code is presented in Code example 4.3. If we did not choose to store the data, we could instead simply have passed the arrays returned by `integrate_solar_system()` directly to the plotting routines. Alternatively, we could make a simple animation of the data, as presented in Code example 4.4.

Storing the animation as a gif file can be achieved by replacing line # 37 in Code example 4.4 by

```
anim.save("anim_gravity.gif", dpi=150, writer=animation.PillowWriter(fps=25))
```

4.4.1.2 Calculating Orbital Elements as Diagnostics

It is often useful to convert the six-dimensional Cartesian coordinates of a two-body system into Kepler orbital elements—semimajor axis, eccentricity, phase, etc. Because this is a common and important conversion, AMUSE offers a special routine to compute the orbital semimajor axis and eccentricity of any two-body system. A more detailed description of the use of Kepler elements, including discussion of some efficiency considerations, is presented in Section 8.3.2.

The two particles for which orbital elements are to be calculated must be stored in a separate particle set containing only those particles. For example, when we consider the solar system calculation in Section 4.4.1 (Code example 3.6), we can declare a subset of local particles `SunEarth` containing the Sun and Earth, respectively.

```

9 import matplotlib.pyplot as plt
10 from matplotlib import animation
11 from amuse.io import read_set_from_file
12 from amuse.units import units
13
14
15 def animate(x, y):
16
17     def update(i):
18         data = np.stack([x[i], y[i]]).T
19         scat.set_offsets(data)
20
21         return (scat,)
22
23     number_of_snaps = len(x)
24     fig = plt.figure(figsize=(8, 8))
25     ax = fig.add_subplot(1, 1, 1)
26     ax.set_xlim((-1.2, 1.2))
27     ax.set_ylim((-1.2, 1.2))
28     ax.set_xlabel("X [au]")
29     ax.set_ylabel("Y [au]")
30
31     colormap = ["#FFFF00", "wheat", "deepskyblue"]
32     size = [40, 20, 20]
33     edgecolor = ["orange", "wheat", "deepskyblue"]
34
35     ax.set_facecolor("black")
36     scat = ax.scatter(x[0], y[0], c=colormap, s=size, edgecolor=edgecolor)
37     anim = animation.FuncAnimation(fig, update, frames=number_of_snaps, interval=30)
38     anim.save("anim_gravity.mp4", dpi=150, writer=animation.FFMpegWriter(fps=25))

```

Code example 4.4: Minimal routine to animate N -body simulation data. Full source code is available in `amuse.examples.anim_gravity`.

```

SunEarth = Particles()
SunEarth.add_particle(particles[0])
SunEarth.add_particle(particles[2])

```

If desired, we could create a separate channel to this particle set:

```
channel_from_to_SunEarth = gravity.particles.new_channel_to(SunEarth)
```

The orbital elements can then be determined by calling

```
orbital_elements = orbital_elements_from_binary(SunEarth, G=constants.G)
```

This function is part of the extended AMUSE package, imported via

```
from amuse.ext.orbital_elements import orbital_elements_from_binary
```

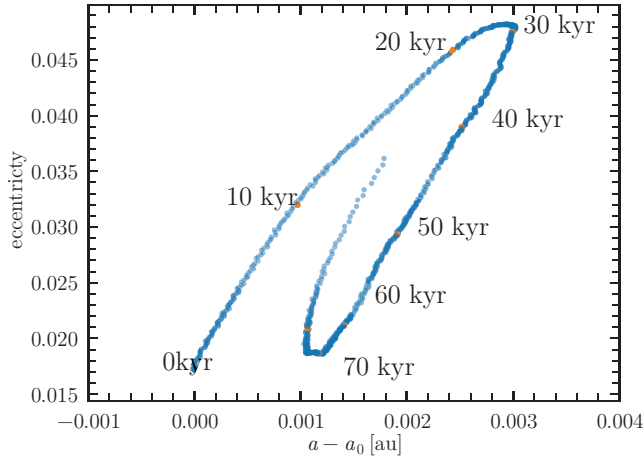


Figure 4.5. Earth’s orbital semimajor axis (x -axis) and eccentricity (y -axis) as functions of time over a period of 80,000 yr. Initial conditions are taken from Table 3.1. The complete code example for the script that produced this figure is in `amuse.examples.earth_orbit_variation`. It should take a minute or so to run on a laptop.

The Kepler package underlying this function is written in standard N -body units and requires some notion of the value of Newton’s constant G , which is provided as an argument. The routine returns a tuple containing the masses of the primary and secondary stars and the six Kepler elements: semimajor axis, eccentricity, inclination, argument of periapsis, line of the ascending node, and true anomaly.

We can integrate the inner solar system and study the variation in Earth’s semimajor axis and eccentricity over an interval of 90,000 years. We present this evolution in Figure 4.5. Instead of initializing the solar system from the simplified initial conditions given earlier in Code example 3.6, we use AMUSE’s built-in generator for any Julian date. To generate an initial realization of the solar system at the Julian date of 2,438,871.5 day, we write

```
particles = new_solar_system(Julian_date=2438871.5 | units.day)
```

Figure 4.6 presents the results of the numerical integration of the solar system over a much longer timescale. We will use this script in Section 4.4.1.3 to compare the variations in the eccentricity of Earth’s orbit versus the temperature measurements at the Vostok station in Antarctica.

4.4.1.3 Temperature Record at Vostok Station

While the cycles in Earth’s orbital elements are intrinsically interesting, it is even more interesting to compare them with observations. Instead of integrating just the innermost two planets, we include all eight planets in the solar system. In order to draw a comparison with observations, we integrate the solar system backward in

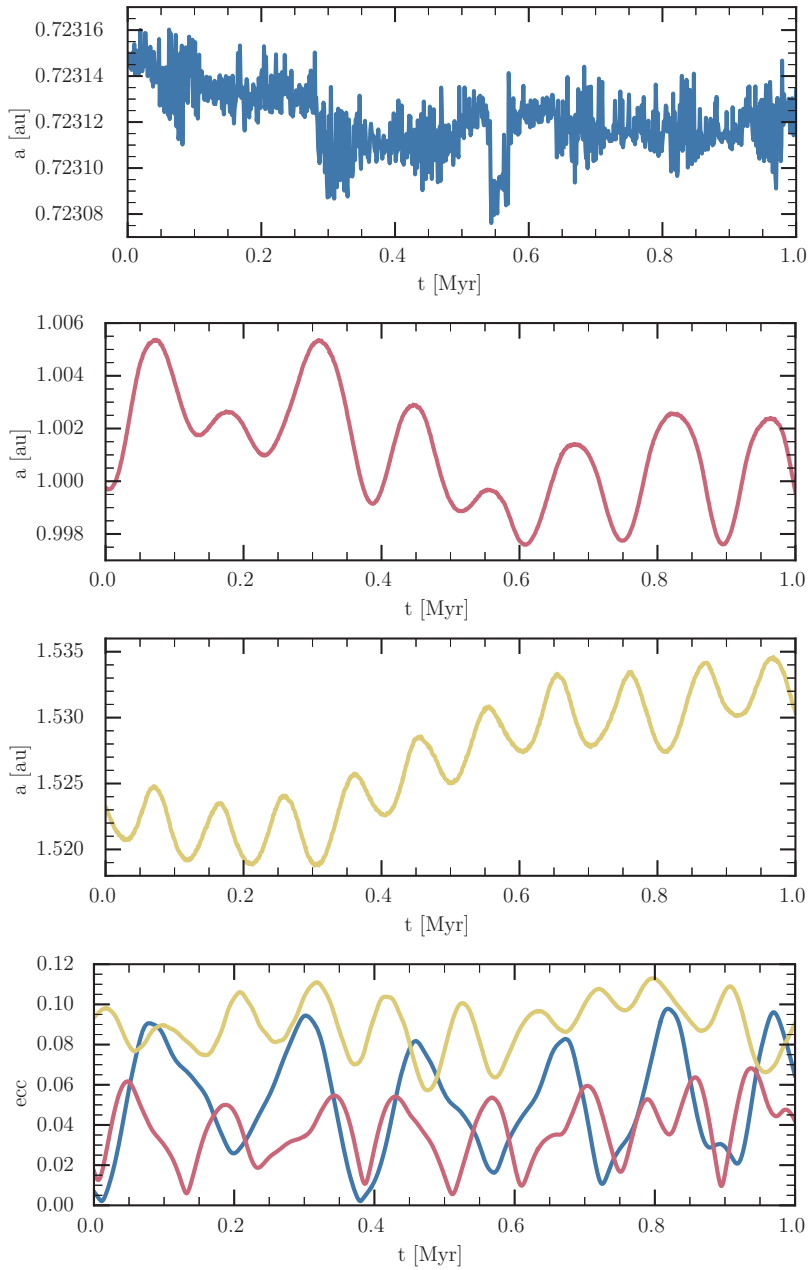


Figure 4.6. From top to bottom, the respective evolutions of the orbital semimajor axes of Venus (blue), Earth (red), and Mars (yellow) for the next million years. The bottom figure gives the orbital eccentricities of the three planets as functions of time, using the same color coding. The integration was performed using a script similar to that presented in Code example 3.7.

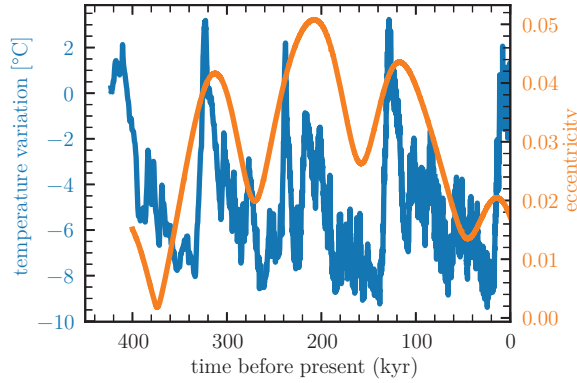


Figure 4.7. (Blue) Historical isotopic temperature record from the Vostok ice core (Jouzel et al. 1987; Petit et al. 1999). (Red) Orbital eccentricity evolution of the Earth over the last 400,000 yr. Integration was performed using `Huayno` with an accuracy parameter $\varepsilon = 10^{-3}$. The full code example of the script that produced this figure is in `amuse.examples.vostok_1999_temperature`. The Vostok ice-core temperature measurements are available via `amuse.examples.export("vostok_1999_temperature.data", ".")`.

time and check the results against the historical temperature record obtained from the ice at Vostok station in Antarctica. The backward calculation can be realized using one of the time-reversible N -body codes in `AMUSE`, or simply by reversing the velocities and integrating forward in time (the latter method was adopted here).

Figure 4.7 shows the measured changes in the temperature at Vostok station and the numerically computed changes in Earth’s orbital eccentricity over the past 450,000 yr. The variations in eccentricity seem to correspond to variations in temperature (and CO_2) in Earth’s atmosphere (Gildor & Tziperman 2000). It is interesting to note that the sharp rise in temperature seems to correlate with a rising eccentricity when it exceeds about 0.035. It has been suggested that these variations in the Earth’s orbital eccentricity drive the 100,000 year ice-age cycle found in the historical temperature record from the Vostok ice core (Milankovitch 1941), but Earth’s obliquity is important too.

4.4.2 Relativistic N -body Integration

Figure 4.8 shows an integration of the first stable three-body solution found by Montgomery (1998). This was the first stable nonhierarchical system to be discovered, and it caused quite a stir in N -body circles at the time. Subsequently, many stable “choreographies” of multiple bodies all following a single orbit were identified. Usually, this system is integrated in the Newtonian regime, but here we also show a solution where the system is relativistic. Even though we only integrate for a short time span, it is clear that the relativistic case the system is considerably less stable than in the Newtonian case.

This example was calculated using `hermite_grx` using the 2.5th order Taylor expansion to the Einstein–Infeld–Hoffman equations of motion with a scaling constant of $\gamma = 0.01c$. The stellar masses were $1 M_\odot$ and the mutual distance 1 au.

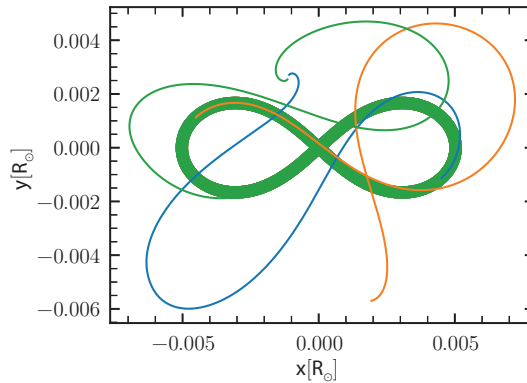


Figure 4.8. Montgomery’s (1998) three-body problem, or the so-called stable figure 8 configuration in Newtonian gravity (thick green curve) for three black holes of $1 M_{\odot}$. The three thin curves (green, orange, and blue) give the trajectories for the same initial configuration but when the velocities of the particles approach the speed of light. The calculations were conducted using `hermite_grx` with a scaling to the speed of light of $0.01c$. Whereas the Newtonian case shows the expected behavior predicted by Montgomery, the relativistic case ends in a merger between two, and eventually all three stars. The script to generate this figure can be found in `amuse.examples.montgomery`.

```

95     n_body_code = Hermitegrx
96     grav = n_body_code(converter)
97     pert = "2.5PN_EIH"
98     grav.parameters.perturbation = pert
99     grav.parameters.integrator = "RegularizedHermite"
100    grav.parameters.dt_param = 0.01
101    grav.parameters.light_speed = relative_light_speed * constants.c
102

```

Code example 4.5: Setting up the relativistic gravitational N -body code to integrate to 2.5th order PN terms with the regularized Hermite integrator. The full script can be found in `amuse.examples.montgomery`.

For a usual speed of light, this system is barely relativistic (represented by the thick green curve). But when we scale the speed of light to $\gamma = 0.01c$, the orbits deviate considerably from their Newtonian trajectories. Interestingly, one naively would expect the system to shrink, rather than the wider excursions as observable in Figure 4.8. In the relativistic case, two of the three stars collide (top middle). The setting up of the code is listed in Code example 4.5.

4.4.3 Fighting Exponential Divergence

All computers make errors. Larger computers just make more errors in a shorter amount of time than smaller computers. Normally, the 16 decimal places we work with in 64 bit IEEE-754 floating-point arithmetic hides the round-off nicely, but the problems may become apparent when dealing with large dynamic range or studying chaotic systems.

When we resolve planetary impact (on a scale of 10^3 km) in the local group of galaxies (100 kpc), the dynamic range exceeds a factor of 10^{14} , leaving only one or two digits for precision. This loss of precision when dealing with a wide dynamic range is mitigated by adopting different codes for different domains. For example, the solar system can be addressed in a dedicated code, whereas the local group is addressed in another code.

We do not have this luxury when studying chaos and can only solve this problem in two ways: either by adopting a statistical approach or using arbitrary precision. The first option is the most common, but the second has interesting possibilities.

Brutus (Boekholt & Portegies Zwart 2015) is a code that handles numerical round-off errors by simply hiding them. This is realized by combining an arbitrary precision library (Fousse et al. 2007) with the Bulirsch–Stoer integration strategy (Bulirsch & Stoer 1964).

The Brutus code is called similarly to any other N -body code in AMUSE, with the exception that it requires two additional parameters for tuning the precision and accuracy of the code. Precision can be tuned with the `word_length`, which is the number of bits with which the floating-point numbers in the code are represented. For regular double precision one would then use 64 bits.

The other parameter, called `bs_tolerance` (Bulirsch–Stoer tolerance), gives the accuracy at which the integration is performed. The smaller this parameter is set, the more accurate the calculation is performed. Although there is no strict linear relation between the `bs_tolerance` (T_{BS}) and the `word_length` (L_w), we found an empirical expression that relates one to the other,

$$L_w = 32 + 4|\log_{10}(T_{BS})|. \quad (4.18)$$

A word length of $L_w = 64$ bits then leads to a Bulirsch–Stoer of $T_{BS} = 10^{-8}$.

The following snippet shows how to call Brutus.

```
gravity = Brutus()
gravity.parameters.bs_tolerance = bs_tolerance
gravity.parameters.word_length = word_length
gravity.parameters.dt_param = dt_param

gravity.particles.add_particles(bodies)
channel_to_framework = gravity.particles.new_channel_to(bodies)
```

The result of a calculation of a virialized three-body Plummer with two massive bodies and one minor object is presented in Figure 4.9. The orbits are integrated for 10 N -body time units, a tolerance of 10^{-16} , and a word length of 96 bits (24 decimal places, rather than the usual 16). A second calculation is performed with a tolerance of only 10^{-3} and a word length of 44 bits (11 decimal places).

The phase-space distance between the two solutions is shown as the solid line presented in Figure 4.10. The Lyapunov timescale (Lyapunov 1892) is given by the reciprocal of the slope of this dashed curve. We fit a slope of ~ 0.844 , which corresponds to a Lyapunov timescale of about 1.2 N -body time units.

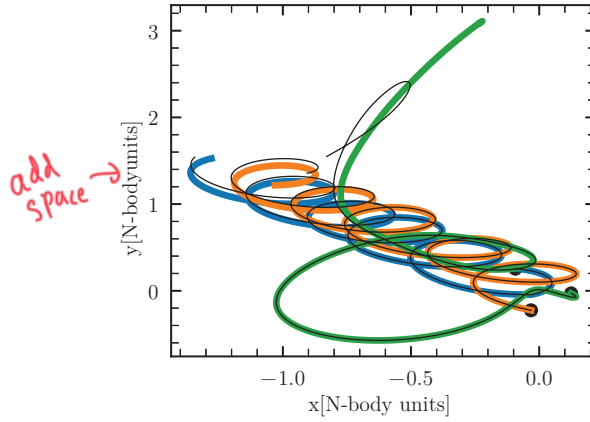


Figure 4.9. Orbital trajectories of a three-body problem (blue, orange, and green curves) calculated with 96-bit precision. The black curves give the result of identical initial conditions but calculated with 24-bit precision. The script, `amuse.examples.brutus`, takes about a minute to run.

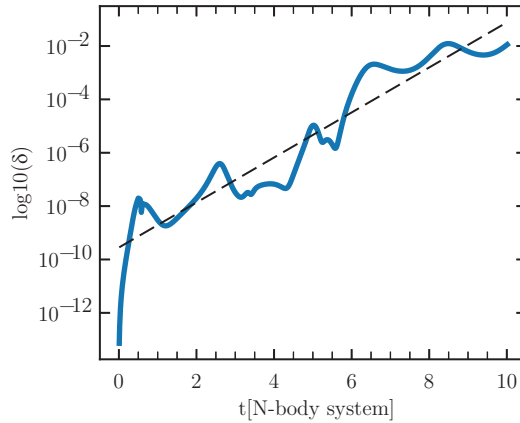


Figure 4.10. Phase-space distance between two calculations with 96-bit and 24-bit precision. The dashed line is a fit. The script to generate this figure can be found in `amuse.examples.brutus`.

Running Brutus requires patience and determination given that it is arguably the slowest N -body code on the planet. However, for studying the chaotic behavior of self-gravitating systems, there is no practical alternative.

Instead of using arbitrary precise floating-point arithmetic, posits (or unum type III) seem to provide a promising alternative for higher precision at lower cost (Gustafson 2015). So far, however, there are no practical hardware implementations of posits, and the few available implementations go to a maximum of 64 bits.

4.4.4 Secular Multiples

When integrating planetary systems, sometimes resolving the individual orbital phases of all planets is simply too time consuming, particularly when the system is

sufficiently stable that an approximate approach can suffice. In these cases, we can orbit-average the equations of motion. The basic idea behind such orbit averaging is the “wire” method, in which each planet is imagined to be spread out over its Keplerian orbit as though on a wire. Along that imaginary wire, the local density is proportional to the time spent at that orbital phase, with higher density near apocenter and lower density near pericenter. This approach can result in a substantial increase in computational speed. `SecularMultiple` is such a code (Hamers & Portegies Zwart 2016) in AMUSE.

In `SecularMultiple`, the secular (i.e. orbit-averaged) evolutions of hierarchical multiple systems are composed of binary orbits, in much the same way as is done in `Sakura` (see Table 4.1 or Section A.8), with an arbitrary number of bodies. For application domains, one can think of a hierarchical triple system with two stars in an “inner” binary orbited by a third body, a planetary system with a central star and several planets, or a hierarchical quintuple system with multiple planets. The code is based on an expansion of the Hamiltonian of the system in terms of ratios of orbital separations, which are assumed to be small. Subsequently, the Hamiltonian is averaged analytically over all orbits and the equations of motion are solved numerically. Speed is the main advantage of this approach: the code runs faster than any direct N -body integrator while still capturing the secular evolution. Individual terms in the expansion can easily be switched on or off to provide a better understanding of their effects on the evolution of the system. A downside is that the system must be hierarchical, since nonsecular effects are not captured. In particular, systems in which mean-motion resonances are important (such as tightly packed planetary systems) should be avoided.

We illustrate `SecularMultiple` for a “2 + 2” quadruple system: two binaries orbiting one another. As in other N -body codes in AMUSE, `SecularMultiple` works with particle sets. However, in addition to specifying parameters for the individual particles, the hierarchy of the system must also be specified. The declaration of particle attributes in a hierarchical quadruple system is illustrated in Code example 4.6. These attributes connect particles to one another. In this case, a body is a particle with a mass, and with attribute `is_binary` set `False`. Binaries are also part of the particles set, but with a `True` value for `is_binary`. Each binary has two children, `child1` and `child2`, indicating other members (bodies or other binaries) of the particle set, as well as attributes describing its orbital elements.

The example script has four bodies, `particles[0]` to `particles[3]`, and three binaries. The first binary $\mathcal{A}$ is `particles[4]`, the second binary $\mathcal{B}$ is `particles[5]`, and the outer orbit $\mathcal{C}$ of the $\mathcal{A}$ - $\mathcal{B}$ binary is `particles[6]`. We adopt masses $1.0 M_{\odot}$ and $0.8 M_{\odot}$ for binary $\mathcal{A}$ with $a_{\mathcal{A}} = 1.0$ au, $e_{\mathcal{A}} = 0.1$, and $i_{\mathcal{A}} = 75^{\circ}$. Binary $\mathcal{B}$ has primary and secondary masses of $1.1 M_{\odot}$ and $0.9 M_{\odot}$, $a_{\mathcal{B}} = 1.2$ au, $e_{\mathcal{B}} = 0.1$, and $i_{\mathcal{B}} = 80^{\circ}$. The outer binary, $\mathcal{C}$, has $a_{\mathcal{C}} = 100$ au, $e_{\mathcal{C}} = 0.3$.

Running `SecularMultiple` is no different from running any other N -body code in AMUSE, using the function `evolve_model()` in the event loop. Figure 4.11 presents the evolution of the orbital separation and eccentricity of the

```

24 def initialize_multiple_system(
25     number_of_bodies,
26     masses,
27     semimajor_axis,
28     eccentricity,
29     inclination,
30     argument_of_pericenter,
31     longitude_of_ascending_node,
32 ):
33     """
34     Initializes a system of multiples
35     """
36
37     number_of_binaries = number_of_bodies - 1
38     particles = Particles(number_of_bodies + number_of_binaries)
39     for index in range(number_of_bodies):
40         particle = particles[index]
41         particle.mass = masses[index]
42         particle.is_binary = False
43         particle.radius = 1.0 | units.RSun
44         particle.child1 = None
45         particle.child2 = None
46
47     for index in range(number_of_binaries):
48         particle = particles[index + number_of_bodies]
49         particle.is_binary = True
50         particle.semimajor_axis = semimajor_axis[index]
51         particle.eccentricity = eccentricity[index]
52         particle.inclination = inclination[index]
53         particle.argument_of_pericenter = argument_of_pericenter[index]
54         particle.longitude_of_ascending_node = longitude_of_ascending_node[index]
55
56         # Specify the '2+2' hierarchy:
57
58         if index == 0:
59             particle.child1 = particles[0]
60             particle.child2 = particles[1]
61         elif index == 1:
62             particle.child1 = particles[2]
63             particle.child2 = particles[3]
64         elif index == 2:
65             particle.child1 = particles[4]
66             particle.child2 = particles[5]
67     binaries = particles[particles.is_binary]
68
69     return particles, binaries

```

Code example 4.6: Snippet to set up a “2+2” hierarchical quadruple system composed of two binaries that orbit each other. The full script can be found in `amuse.examples.hierarchical_quadruple`

various components of this quadruple system. The top panel shows the semimajor axes (solid curves) and periapsis distances (dotted lines) of the three orbits $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$; the bottom panel shows the inclinations of $\mathcal{A}$ and $\mathcal{B}$ relative to $\mathcal{C}$. High Lidov–Kozai-like eccentricity oscillations occur in the $\mathcal{A}$ and $\mathcal{B}$ orbits, although they are more complex than the equivalent isolated hierarchical triple systems and the

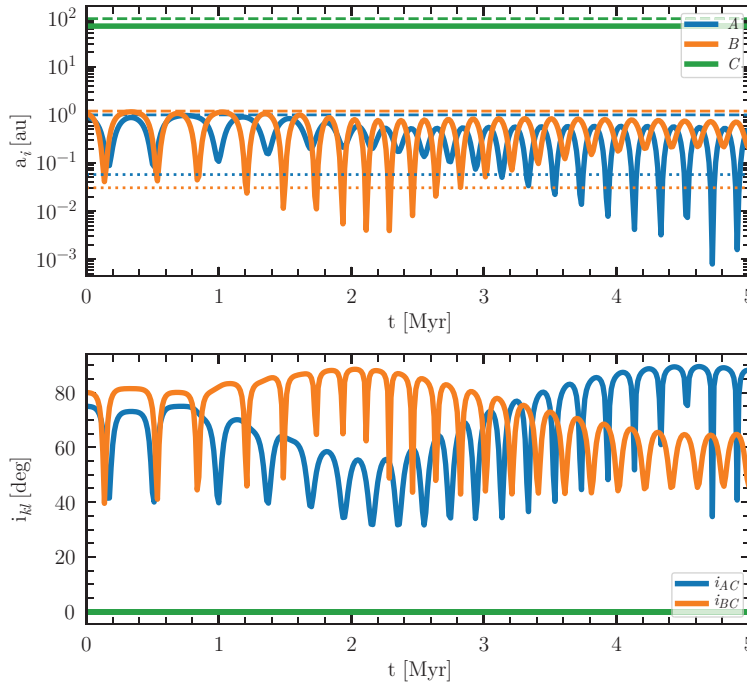


Figure 4.11. Example evolution of a 2 + 2 quadruple system. Top panel: The semimajor axes (dashed lines) and periapsis distances (solid curves) of orbits $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$ as a function of time. Bottom panel: The inclinations of $\mathcal{A}$ and $\mathcal{B}$ relative to $\mathcal{C}$. Canonical values expected for Lidov–Kozai oscillations in the three-body test-particle quadrupole-order approximation are shown in the top panel as dotted lines. The dynamics are more complex in this quadruple system, compared to the situation of two uncoupled triple systems. The script that produced this figure is `amuse.examples.hierarchical_quadruple`. It takes about half a minute to run.

maximum eccentricities are considerably higher. (To study the difference due to the fourth body in the system, we can easily reduce the semimajor axis of one of the binaries $\mathcal{A}$ or $\mathcal{B}$ by a factor of 100, effectively reducing it to a point mass. This quenches the effect of one of them being a binary and recovers the classic Lidov–Kozai effect in the remaining triple system.)

In addition to point-mass secular Newtonian dynamics, other physical processes can be included in `SecularMultiples`. Such as PN effects up to and including order 2.5, or tidal evolution (including misaligned spins). The Mardling & Aarseth (2001) stability criterion for hierarchical triples is also implemented and checked.

4.4.5 Merging Galaxies

There are at least three methods whereby gravity is calculated in SPH codes. These include direct summation, the tree algorithm, and self-consistent mean fields using fast Fourier transforms (Hockney & Eastwood 1988). Direct summation has the advantage of allowing accurate energy conservation, which can be important for problems near a stability limit (e.g., for a binary star system near its innermost

stable circular orbit). These direct-summation methods are typically employed on GPUs using the same algorithms as for N -body codes (Portegies Zwart et al. 2007).

The most common engine for computing self-gravity in an SPH code is a tree code. Such codes are well suited to simulating galaxy mergers. In Section 3.2, we discussed how to generate a galaxy model using Halogen (Zemp et al. 2008; Zemp 2014). Here, we will use GalactICs (Kuijken & Dubinski 1995), which is designed to set up a self-consistent galaxy model with a disk, bulge, and dark halo. In principle, this code can be used to construct realistic models with both gas and particles for the Milky Way, Andromeda, or any other galaxy, but it often requires some delicate tuning to get the proper structure.

We use GalactICs to generate two identical disk galaxies with `n_halo` particles in each halo, `n_bulge` particles in each bulge, and `n_disk` particles in each disk. We eventually scale the models to virial equilibrium. The first galaxy is created as follows:

```
converter = nbody_system.nbody_to_si(mass_galaxy, radius_galaxy)
galaxy1 = new_galactics_model(
    n_halo,
    converter,
    do_scale=True,
    bulge_number_of_particles=n_bulge,
    disk_number_of_particles=n_disk,
)
```

The second galaxy is constructed by simply making a copy of the first. Both galaxies are placed in an interacting orbit; the hydrodynamics solver is then provided with the particles and integrates the equations of motion without taking hydrodynamics into account.

```
converter = nbody_system.nbody_to_si(1.0e12 | units.MSun, 100 | units.kpc)
dynamics = Gadget2(converter, number_of_workers=4)
dynamics.parameters.epsilon_squared = (100 | units.parsec)**2
set1 = dynamics.particles.add_particles(galaxy1)
set2 = dynamics.particles.add_particles(galaxy2)
dynamics.particles.move_to_center()
```

For plotting purposes, we want to keep track of only the disk and bulge particles. This is accomplished by selecting only those particles added after the halo was generated (the order of creation is halo, bulge/disk). For this reason, we save pointers to the particles in the running code in the local parameters `set1` and `set2` for the first and second galaxies, respectively. Here, we take advantage of the fact that `add_particles` always returns a pointer to the last object added (in this case the bulge/disk particles).

Figure 4.12 presents two snapshots of the interacting galaxies, one at the start of the simulation and the other 200 Myr later when the two galaxies have just passed each other. The two galaxies are identical copies, but they are rotated using the particle-set attribute `rotate`. For the galaxy on the left in Figure 4.12 (red), this is realized with

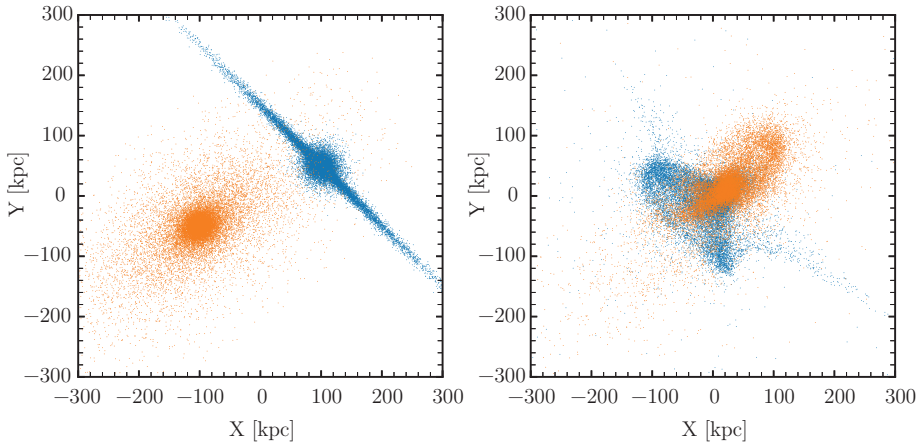


Figure 4.12. Collision between two galaxies, just before they interact (left) and 200 Myr later (right), when both galaxies have been distorted by the interaction. The calculation was performed using `Gadget2` (see Chapter 6 and Springel 2000, 2005) as a gravity solver, without hydrodynamics. Both galaxies are made of 20,000 halo particles, with 10,000 for each the bulge and the disk. In this figure, we only show the bulge and disk particles. The simulation was performed using `amuse.examples.merge_two_galaxies`. It should take a couple of minutes to run on a laptop.

```
galaxy1.rotate(0.0, numpy.pi/2, numpy.pi/4)
```

The galaxies are subsequently translated to a new position about 200 kpc apart and given a velocity toward each other of about 10 km s^{-1} . We refrain here from a detailed analysis of the resulting merger, but it may be rewarding to simulate the cosmic collision between the Milky Way and the Andromeda galaxy.

4.5 Choosing the Right Code

It is not always trivial to choose the right code for your project. There is generally an optimal choice, but one can wonder if there is a wrong choice. Deciding on what code to select is not trivial, and the optimization matrix may be complex. The arguments that tip the balance in favor of one code can be based on the physics, precision, accuracy, long-term stability, energy error, angular momentum conservation, performance, and so on.

The first question to ask is to what extent the gravity in your problem should be coupled to another physical domain. We cannot really make a firm recommendation because each code can be coupled to other solvers in AMUSE (see Section 8 and Chapter 9), although there are a few details to consider.

Each code in AMUSE has an application domain. Once the calculations leave this domain, the code stops working in some way. It could crash, but worse it could become inefficient or continue without producing sensible results. To mitigate this problem, we have introduced what we call *stopping conditions*.

4.5.1 Stopping Conditions

Stopping conditions are criteria which, once reached, allow the particular code triggering the condition to stop in an appropriate state. Control is then returned to the AMUSE script, in which action may be taken. These actions include stopping the particular code, handing the data to another already existing code, storing data and performing diagnostics, or starting a new code to perform further calculations. Stopping conditions are a fundamental and powerful way to mitigate all sorts of problems that may arise when combining different solvers operating on a wide variety of domains. They are fundamental to AMUSE, and each domain has its own set of stopping conditions.

The most common stopping conditions for gravity are listed in Table 4.2. These include detection of collisions, escapers, if an interaction is still taking place, and binary (pair) formation. There are also possible limits that can be imposed using stopping conditions, such as the number of steps in the integrator or a time-out for the calculation.

A stopping condition, for example to detect a collisions, can be used as follows:

```
gravity = Ph4()
collision_detection = gravity.stopping_conditions.collision_detection
collision_detection.enable()
```

Here, the N -body code will stop as soon as two stars approach each other within the sum of their radii (however defined). The interrupt is called in the event loop where the N -body code is evolved,

Table 4.2. Overview of the Stopping Conditions Available in Most N -body Codes

<code>code.stopping_conditions.</code>	Default Value	Default Unit
<code>...number_of_steps</code>	1	None
<code>...timeout</code>	4.0	s
<code>...collision_detection</code>	bool	False
<code>...escaper_detection</code>	bool	False
<code>...interaction_over_detection</code>	bool	False
<code>...pair_detection</code>	bool	False

```
while gravity.model_time < time_end:
    gravity.evolve_model(gravity.model_time + time_step)
    if collision_detection.is_set():
        resolve_collision(collision_detection, gravity, converter)
```

The appropriate information about the event can be queried from `collision_detection` through accessor functions. If particles are associated with the event, these are stored as a particle set in

```
collision_detection.particles
```

and can be used as such.

Note that although the state of the N -body code is consistent, it will not have completed its evolution to the appropriate end time (`gravity.model_time + time_step`), but has stopped at same time between `gravity.model_time` and `gravity.model_time + time_step`). In this example, the routine `resolve_collision()` resolves the collision by merging the masses of the two colliding stars, and communicates their center of mass properties to the new particle that replaces the two colliding particles. In principle, this can be realized in a few lines of code, or one may invoke a hydrodynamics solver to effectively resolve the stellar collisions.

4.6 Validation

Quickly prototyping a small script that runs a large scientific simulation is easy with AMUSE, but how do you guarantee that everything works correctly? You cannot, and we cannot either. However, we can at least guarantee that the coupling of the code is done correctly and that the units are properly converted and transferred.

Some codes are easier to validate than others. Direct gravitational solvers have to conserve energy, at least if they are run as isolated, standalone modules. Checking for energy conservation is therefore always a good thing to do. For stellar evolution, there is no conserved energy, but you would surely like the total mass of the star plus the total mass lost in a wind and a supernova to be conserved. Each methodology has its own conserved quantities. As a researcher and user of any scientific code, you should always explicitly check them.

Time-stepping in any given code is generally managed internally, but as a user you can in principle change the time steps as you like. However, in addition to the fact that we do not recommend fiddling with the community source code, we also do not recommend overriding the safety settings for things like time step control. Some of the community modules are very sensitive to such changes and may simply fail, but others will always produce some output. These latter codes are the most dangerous: “garbage in, garbage out.” If a code receives completely wrong information, in the form of nonsensical initial conditions or parameter settings, then the best thing that can happen is that it crashes. The worst outcome is that the code will continue to run, producing reasonable-looking nonsense.

4.6.1 Error Propagation and Validation

The solar system is about 4.5 Gyr old, which means that Earth has made more than four billion orbits around the Sun. During each integration step in a simulation, the program makes tiny mistakes (e.g., round-off errors at about the 15th digit, or larger errors due to algorithmic limitations), which slowly accumulate with time. The accumulated energy error may eventually grow to exceed the total energy of the system, at which point we would be forced to conclude that the integration has failed to represent the physical system we are trying to model.

For lack of a satisfactory alternative, the energy error is often used as the sole diagnostic for quantifying the quality of an N -body integration—but beware: a gravitational system (probably) responds exponentially to small errors, drifting away from whatever the true solution is and probably rendering the numerically determined position and velocity of any particle meaningless. Even worse are the small phase-space errors introduced by the integrator which, due to the chaotic nature of the system, grow exponentially. This causes your system to quickly diverge from what you intended to calculate, and you will not notice it because the energy error may remain small.

The error in the energy may grow randomly (according to Brouwer’s law; see Section 4.2.3) or, more likely, systematically and slowly. Therefore, the energy error is sometimes a rather poor indicator of the quality of an N -body integration (partly for reasons discussed in Section 4.1.4.1), but so long as it does not grow beyond a certain fraction of the total energy, the result may be sufficiently accurate for scientific interpretation.

The boundary above which we consider the energy error acceptable is quite arbitrary and context-dependent. If no number is quoted, you should, as a scientist, doubt the quality of the calculation (even though the animations and figures may look great). As a rule of thumb, for direct N -body simulations, the cumulative energy error over the duration of the calculation should remain below 1% of the total energy of the entire system, and below 10^{-4} of the total energy of the system over an interval of one dynamical time. Many researchers would regard these limits as far too high.

4.6.1.1 Error Behavior and Analysis

The errors in N -body integrations can be measured objectively by checking conserved quantities, such as energy and angular momentum. In Code example 4.1, we demonstrated how energy conservation can be checked at runtime. This provides a useful first-order diagnostic, although for the solar system (for example), it informs you mainly about the integration errors for the most massive bodies. It is far less informative about errors in the integration of minor bodies.

The error behavior of N -body codes depends both on the algorithm used to calculate the forces and how the equations of motion are integrated. Figure 4.13 shows the relative energy error for three N -body solvers—a (fourth-order) direct-summation Hermite solver, a similar high-order solver but including the Einstein–Infeld–Hoffman equations of motion for general relativity, and a (second-order)

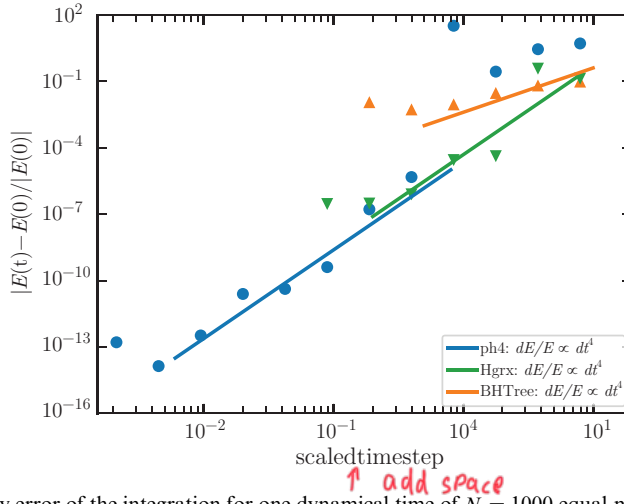


Figure 4.13. Energy error of the integration for one dynamical time of $N = 1000$ equal-mass particles initially distributed in a Plummer sphere. The integration was carried out using the direct N -body integrator ph4 (blue bullets), Hermite_GRX (green triangles), and with the BHTree tree code (orange triangles). The three solid lines show the global trends in energy conservation of the N -body codes (these are not fits). The Hermite_GRX run was performed for only 0.1 N -body time units, to reduce the computational cost. The script to generate this figure can be found in `amuse.examples.run_nbody_precision`. The ph4 and BHTree runs take a few seconds each, but the runs with Hermite_GRX are considerably more expensive, taking about an hour to run on a laptop.

Barnes–Hut tree code with a Leapfrog solver. The general relativistic run was performed with a speed of light one hundred times higher than the maximum velocity of any of the particles, which is rather arbitrary, but demonstrates the point that the scaling is identical to the Newtonian fourth-order integrator.

Each solver behaves somewhat differently, but this is not unexpected. The direct N -body integration can reach very small energy errors, below 10^{-13} . For smaller values of the time-step parameter $\eta \lesssim 0.005$, the error starts increasing again. From this point, numerical round-off starts to drive the energy error in the calculation. We refer to Press et al. (1992), which excellently quantifies this point. For a time step parameter of unity (and larger), the error suddenly increases to a value of $\mathcal{O}(1)$. Once the time step parameter becomes too large (apparently around order unity), the integrator is unable to follow individual trajectories accurately. The integrator does not break completely, but fails to acquire convergence in the solution. This is reflected in the large energy error, and is one of the reasons why it is important to always check the energy conservation in runs.

The relativistic integrator Hermite_GRX shows a similar behavior as the Newtonian integrator, but it is much more expensive to run (the force in general relativity also depends on the velocity and therefore scales $\propto N^3$). The numerical round-off in the relativistic equations of motion starts to become noticeable at a much larger value of $\eta \lesssim 0.1$ compared to the Newtonian calculation, because the number of operations per performance calculation is orders of magnitude larger. It is therefore not very useful to run relativistic calculations with extremely small values

of η (except, of course, when one runs with an arbitrarily precise integrator, such as Brutus).

With the tree code, the error saturates because the error at short time steps is dominated by the tree force evaluation algorithm, not by the integration scheme. However, this comparison is unfair because in BHTree this parameter is defined differently. In BHTree, the time step is either an absolute fraction of the local crossing time (when running in dimensionless N -body units) or it is an absolute value in units of (millions of) years. In this case, the calculations are performed using dimensionless N -body units, and η for BHTree should be interpreted as a fraction of the dynamical crossing time. The other parameter that can be used to tune the precision performance of a tree code such as BHTree is the opening angle θ . In this example, we used the default value of $\theta = 0.75$.

4.7 Assignments

4.7.1 Orbital Trajectories

Take the initial conditions for the solar system (Table 3.1) and integrate the planets' orbits for 100 yr with the integrator of your choice (Table 4.1). Explain why you have selected this integrator, and discuss the effect of using another integrator.

1. Draw graphs of the energy error as a function of time, and of the change in the planet's semimajor axis, da/a , as a function of time.
2. Draw a graph of the evolution of the semimajor axes and eccentricities of Venus, Earth, and Mars, and compare them with the results in Figure 4.6.
3. Run the same initial conditions with two other integrators, and explain the differences in the results.

Compare the evolution of the semimajor axes and eccentricities for Venus, Earth, and Mars with those presented in Figure 4.6. In what quantity are the differences most pronounced?

4.7.2 Vostok

In Section 4.4.1.3, we discussed the variation in the eccentricity of Earth's orbit over the last 450,000 yr. Although the variations in the average temperature measured at the Vostok station are not entirely attributable to the apparent cycle in Earth's eccentricity, it is evident that some periodicity appears in Earth's orbit.

These variations are probably due to the influence of only a few (maybe only one) other planets. It is unlikely that Mercury or Neptune drives this cycle, but the N -body code we used to create Figure 4.7 can help us find out which planet is responsible for the variations in Earth's orbit.

Perform this analysis and find the planet that dominates the evolution of Earth's orbit around the Sun.

4.7.3 Dynamical Binary Formation

- (a) In cluster dynamics, it is common to define an energy scale, called kT (as in thermodynamics), by $K = 3/2NkT$, where N is the number of stars and K

is the total kinetic energy of the system. This turns out also to be an interesting energy threshold for binary systems (see Section 4.1.3.2). Run an N -body simulation with $N = 100$ stars using a code of your choice (see Table 4.1), until the appearance of a binary with a binding energy exceeding $100 kT$. You can take Code example 4.1 as a basis for the N -body simulation. The simplest way to do this is to run in steps of (say) one dynamical time, and compute the relative binding energies of all pairs of stars at the end of each step. This is fairly inefficient, but infrequent and fast enough with 100 stars that it should be doable in Python.

Be careful here because several of the N -body codes in AMUSE use softened potentials and are unable to reach such hard binaries, while others will have great difficulty integrating hard binaries.

Use a Plummer distribution as the initial density profile and take all the masses to be the same.

- (b) Perform the run with a different N -body code, using exactly the same initial realization of the initial conditions.
- (c) Is the moment at which the first $100 kT$ binary appears the same, or does it depend on the code?
- (d) What changes when a mass function is introduced? Adopt a Salpeter mass function (see Equation (3.1)) with $\alpha = -2.35$ (Salpeter 1955). Remember to rescale the system to virial equilibrium (see Section 4.3.1) after setting the stellar masses.

Explain why the system should be rescaled to virial equilibrium.

In an N -body integration performed in dimensionless N -body units, the only parameter to specify for the mass function is the range over which the masses should be varied (the total mass is normalized), assuming that the power-law slope remains the same. How does the moment of the formation of the first hard binary depend on the range of masses?

4.7.4 L_1 Lagrangian Point

Lagrangian points in a self-gravitating system of two bodies are locations at which there is either a peak or a saddle point in the effective potential surface (de Lagrange 1772) in the frame rotating with the bodies. An N -body code can easily be adapted to draw such an equipotential surface. Figure 4.14 presents an example for the case of the Sun and a $0.2 M_\odot$ mass companion at a distance of 1 au. The script to generate these equipotential surfaces can easily be adapted to any binary system, including those with nonzero eccentricity.

Interplanetary travel is expensive in terms of time and fuel. Moving through Lagrange points provides an efficient way to travel from one celestial body to another (although they can also lead to objects becoming trapped in a local orbit, as is the case with Jupiter's Trojan satellites). In Figure 4.14, the most efficient way to travel from the star to the left to the one to the right is through L_1 . This trajectory can be calculated most easily by starting at L_1 and allowing a test mass to “fall” into the potential well to the left or the right.

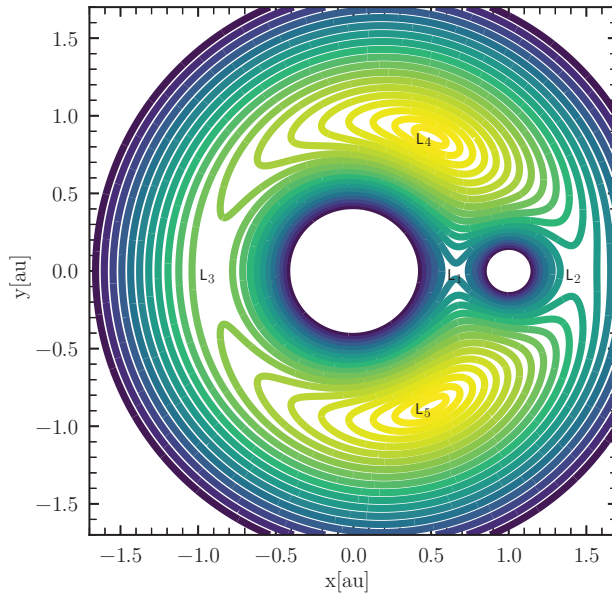


Figure 4.14. Equipotential surfaces for a two-body system. In this example, we adopted a primary mass of $1 M_{\odot}$ with a $0.2 M_{\odot}$ companion in a circular orbit at a distance of 1 au (to the right). The classical five Lagrange points are indicated by $L_1 - L_5$. The script to generate this figure can be found in `amuse.examples.lagrange_points`.

- Calculate the most energy-efficient orbit from Earth to the Moon in an isolated system, but with Earth and Moon on the proper relative orbit. Plot this trajectory in Cartesian coordinates.
- Repeat the calculation for Earth and the Moon in orbit around the Sun, with the planet Jupiter added to further perturb the system. Are the two trajectories the same?

Hint: Start by initializing a particle on the first Lagrangian point of the Earth–Moon system and give it a small velocity toward Earth. Repeat with a small velocity in the direction of the Moon.

4.7.5 Virial Equilibrium

- Calculate the evolution of the virial ratio for an N -body system with a mass function between 0.1 and $100 M_{\odot}$ using the codes `ph4`, `Huayno`, and `BHTree`, starting with an initial virial ratio of $Q = 0.2$ (Section 4.3.1), and continue until the system virializes. Perform the calculation with $N = 100$, increasing by factors of 2 up to $N = 1600$, and make a plot of the first moment in time for which $Q = 0.5$ as a function of N for each of the three codes.

The virial ratio will oscillate due to random close encounters, but the virialization timescale can be estimated by studying the way in which the oscillations damp out and the virial ratio approaches its equilibrium value of 0.5.

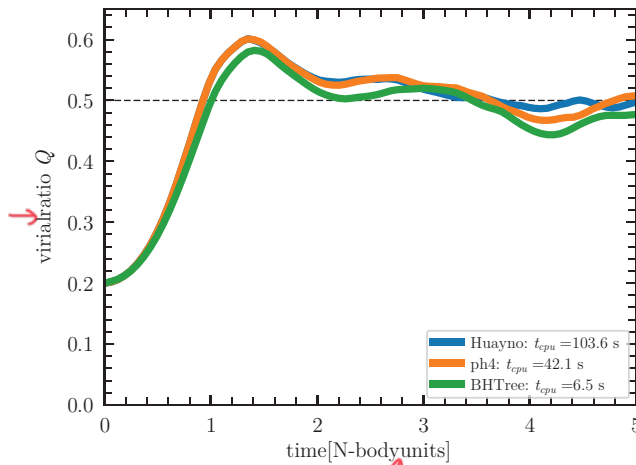


Figure 4.15. Virial ratio Q as a function of time for 1000 equal-mass particles initially distributed in a Plummer sphere with $Q = 0.2$. The integrations were performed using ph4 (blue line), Huayno (red), and BHTree (yellow). The script that produced this figure is in `amuse.examples.gravity_to_virial`. It should run in a couple of minutes on a laptop.

- (b) Figure 4.15 presents an example $q(t)$ for three N -body codes, using 1000 stars of equal mass.

Answer the following questions:

- (i) Why does the system with equal-mass stars virialize on a timescale of about an N -body time unit?
- (ii) What is the effect of introducing a mass function on the virialization timescale?
- (iii) Why does the way in which the system virializes depend on the code?
- (iv) What is the effect of increasing the number of stars on the virialization timescale?
- (v) Why do the two direct N -body systems remain slightly above virial after about 1 N -body time unit?

References

- Aarseth, S. A. 2003, *Gravitational N-body Simulations* (Cambridge: Cambridge: Cambridge Univ. Press)
- Aarseth, S. J. 1963, [MNRAS](#), **126**, 223
- Aarseth, S. J. 1985, *Multiple Time Scales*, ed. J. U. Brackbill, & B. I. Cohen (New York: Academic Press) 377
- Abbott, B. P., Abbott, R., Abbott, T. D., et al. 2016, [PhRvL](#), **116**, 241102
- Ambrosiano, J., Greengard, L., & Rokhlin, V. 1988, [CoPhC](#), **48**, 117
- Anders, P., Baumgardt, H., Gaburov, E., & Portegies Zwart, S. 2012, [MNRAS](#), **421**, 3557
- Antonov, V. A. 1962, *Solution of the Problem of Stability of Stellar System Emdenas Density Law and the Spherical Distribution of Velocities* (Leningrad: Vestnik Leningradskogo Universiteta)

- Babuška, I. 1969, Proc. IFIP Congress Information Processing 68, ed. A. J. H. Morrell (Amsterdam: North-Holland) 11
- Barnes, J., & Hut, P. 1986, [Natur](#), **324**, 446
- Bédorf, J., Gaburov, E., & Portegies Zwart, S. 2012, [JCoPh](#), **231**, 2825
- Bédorf, J., Gaburov, E., Fujii, M. S., et al. 2014, [Proc. of the Int. Conf. for High Performance Computing, Networking, Storage and Analysis, SC '14](#) (Piscataway, NJ: IEEE) 54
- Bettwieser, E., & Sugimoto, D. 1984, [MNRAS](#), **208**, 493
- Binney, J., & Tremaine, S. 2008, Galactic Dynamics (2nd ed.; Princeton: Princeton Univ. Press)
- Boekholt, T., & Portegies Zwart, S. 2015, [ComAC](#), **2**, 2
- Brouwer, D. 1937, [AJ](#), **46**, 149
- Bulirsch, R., & Stoer, J. 1964, [NuMat](#), **6**, 413
- Cai, M. X., Portegies Zwart, S., Kouwenhoven, M. B. N., & Spurzem, R. 2019, [MNRAS](#), **489**, 4311
- Capuzzo-Dolcetta, R., Spera, M., & Punzo, D. 2013, [JCoPh](#), **236**, 580
- Chambers, J. E. 1999, [MNRAS](#), **304**, 793
- Chambers, J. E., & Migliorini, F. 1997, AAS/DPS Meeting, [29](#), 27.06
- Chandrasekhar, S. 1943, [ApJ](#), **97**, 255
- Chernoff, D. F., & Weinberg, M. D. 1990, [ApJ](#), **351**, 121
- Cohn, H. 1980, [ApJ](#), **242**, 765
- Collins, G. W. 1978, The Virial Theorem in Stellar Astrophysics (Tucson, AZ: Pachart Publication)
- de Lagrange, J.-L. 1772, Essai sur le Problème des Trois Corps (Paris: Royal Academy of Sciences)
- Dehnen, W. 2014a, [ComAC](#), **1**, 1
- Dehnen, W. 2014b, gyrfalcON: N-body Code, Astrophysics Source Code Library, ascl:[1402.031](#)
- Duncan, M. J., Levison, H. F., & Lee, M. H. 1998, [AJ](#), **116**, 2067
- Einstein, A. 1916, [AnP](#), **354**, 769
- Einstein, A., Infeld, L., & Hoffmann, B. 1938, [AnMat](#), **39**, 65
- Fellhauer, M., & Lin, D. N. C. 2007, [MNRAS](#), **375**, 604
- Fousse, L., Hanrot, G., Lefèvre, V., Pélissier, P., & Zimmermann, P. 2007, [ACM Trans. Math. Softw](#), **33**, 13–es
- Fukushige, T., & Heggie, D. C. 1995, [MNRAS](#), **276**, 206
- Gaburov, E., Bédorf, J., & Portegies Zwart, S. 2010, OCTGRAV: Sparse Octree Gravitational N-body Code on Graphics Processing Units, Astrophysics Source Code Library, ascl:[1010.048](#)
- Giersz, M., & Heggie, D. C. 1994, [MNRAS](#), **268**, 257
- Giersz, M., & Heggie, D. C. 1996, [MNRAS](#), **279**, 1037
- Gildor, H., & Tziperman, E. 2000, [PalOc](#), **15**, 605
- Gnedin, N. Y., Glover, S. G. O., Klessen, R. S., & Springel, V. 2015, [Star Formation in Galaxy Evolution: Connecting Numerical Models to Reality](#), ed. Y. Revaz, P. Jablonka, R. Teyssier., L. Mayer, et al. (Berlin: Springer)
- Greengard, L., & Rokhlin, V. 1987, [JCoPh](#), **73**, 325
- Gustafson, J. L. 2015, [The End of Error: Unum Computing](#) (Boca Raton, FL: CRC Press)
- Hamers, A. S., & Portegies Zwart, S. F. 2016, [MNRAS](#), **459**, 2827
- Harfst, S., Gualandris, A., Merritt, D., et al. 2007, [NewA](#), **12**, 357
- Heggie, D. C. 1975, [MNRAS](#), **173**, 729
- Heggie, D. C., & Mathieu, R. D. 1986, The Use of Supercomputers in Stellar Dynamics 267, ed. P. Hut, & S. L. W. McMillan; Berlin: Springer) [233](#)

- Heggie, D., & Hut, P. 2003, *The Gravitational Million-Body Problem: A Multidisciplinary Approach to Star Cluster Dynamics* (Cambridge: Cambridge Univ. Press)
- Hellström, C., & Mikkola, S. 2010, [CeMDA](#), **106**, 143
- Hénon, M. 1961, *AnAp*, **24**, 369
- Hénon, M. 1972, *IAU Colloq. 10: Gravitational N-Body Problem* ed. M. Lecar (Dordrecht: Reidel) 406
- Hénon, M. 1975, *IAU Symp. 69, Dynamics of the Solar Systems* ed. A. Hayli (Dordrecht: Reidel) 133
- Hénon, M. H. 1971, [Ap&SS](#), **14**, 151
- Hills, J. G. 1975, [AJ](#), **80**, 809
- Hockney, R. W., & Eastwood, J. W. 1988, *Computer Simulation Using Particles* (Bristol: Hilger)
- Hulse, R. A., & Taylor, J. H. 1975, [ApJL](#), **195**, L51
- Hut, P., Makino, J., & McMillan, S. 1995, [ApJL](#), **443**, L93
- Inagaki, S., & Saslaw, W. C. 1985, [ApJ](#), **292**, 339
- Iwasawa, M., Portegies Zwart, S., & Makino, J. 2015, [ComAC](#), **2**, 6
- Iwasawa, M., Oshino, S., Fujii, M. S., & Hori, Y. 2017, [PASJ](#), **69**, 81
- Jänes, J., Pelupessy, I., & Portegies Zwart, S. 2014, [A&A](#), **570**, A20
- Jouzel, J., Genthon, C., Lorius, C., Petit, J. R., & Barkov, N. I. 1987, [Natur](#), **329**, 403
- Kahan, W. 1965, [Commun. ACM](#), **8**, 40
- King, I. R. 1966, [AJ](#), **71**, 64
- Kozai, Y. 1962, [AJ](#), **67**, 591
- Kuijken, K., & Dubinski, J. 1995, [MNRAS](#), **277**, 1341
- Laskar, J., & Robutel, P. 2001, [CeMDA](#), **80**, 39
- Lee, H. M., & Ostriker, J. P. 1987, [ApJ](#), **322**, 123
- Lidov, M. L. 1962, [Planet. Space Sci.](#), **9**, 719
- Liu, L., Wu, X., Huang, G., & Liu, F. 2016, [MNRAS](#), **459**, 1968
- Lyapunov, A. M. 1892, *Commun. Math. Soc. Krakow*, **2**, 1
- Makino, J. 1991, [ApJ](#), **369**, 200
- Makino, J. 1996, [ApJ](#), **471**, 796
- Makino, J., & Aarseth, S. J. 1992, [PASJ](#), **44**, 141
- Mardling, R. A., & Aarseth, S. J. 2001, [MNRAS](#), **321**, 398
- Maxwell, J. C. 1860, [PMag](#), **19**, 19
- McMillan, S. L. W., & Aarseth, S. J. 1993, [ApJ](#), **414**, 200
- Mikkola, S. 1983, [MNRAS](#), **203**, 1107
- Mikkola, S., & Merritt, D. 2008, [AJ](#), **135**, 2398
- Milankovitch, M. 1941, *Zavod za Udzenike i Nastavna Sredstva: Kanon der Erdbestrahlung und seine Anwendung auf das Eiszeiten-problem* (Belgrade: Königlich Serbische Akademie) (English Translation, 1998, *Canon of Insolation and the Ice Age Problem*)
- Milgrom, M. 1983, [ApJ](#), **270**, 365
- Montgomery, R. 1998, [Nonli](#), **11**, 363
- Naoz, S. 2016, [ARA&A](#), **54**, 441
- Newton, I. 1687, *Philosophiae Naturalis Principia Mathematica* (**Vol 1;** London: Royal Society)
- Nitadori, K., & Makino, J. 2008, [NewA](#), **13**, 498
- Oshino, S., Funato, Y., & Makino, J. 2011, [PASJ](#), **63**, 881
- Peters, P. C. 1964, [PhRv](#), **136**, 1224
- Petit, J. R., Jouzel, J., Raynaud, D., et al. 1999, [Natur](#), **399**, 429
- Plummer, H. C. 1911, [MNRAS](#), **71**, 460

- Poisson, E., & Will, C. M. 2014, Gravity (Cambridge: Cambridge Univ. Press)
- Portegies Zwart, S. F., Boekholt, T. C. N., Por, E. H., Hamers, A. S., & McMillan, S. L. W. 2022, [A&A](#), **659**, [A86](#)
- Portegies Zwart, S. F., & Boekholt, T. C. N. 2018, [MNRAS](#), **478**, [160](#)
- Portegies Zwart, S. F., & McMillan, S. L. W. 2002, [ApJ](#), **576**, [899](#)
- Portegies Zwart, S. F., Belleman, R. G., & Geldof, P. M. 2007, [NewA](#), **12**, [641](#)
- Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. 1992, Numerical Recipes in C: The Art of Scientific Computing (Cambridge: Cambridge Univ. Press)
- Rauch, K. P., & Tremaine, S. 1996, [NewA](#), **1**, [149](#)
- Rein, H., & Liu, S. F. 2012, [A&A](#), **537**, [A128](#)
- Rein, H., & Liu, S.-F. 2011, REBOUND: Multi-purpose N-body Code for Collisional Dynamics, Astrophysics Source Code Library, ascl:[1110.016](#)
- Rodriguez, C. L., Morscher, M., Wang, L., et al. 2016, [MNRAS](#), **463**, [2109](#)
- Saha, P., & Tremaine, S. 1992, [AJ](#), **104**, [1633](#)
- Salpeter, E. E. 1955, [ApJ](#), **121**, [161](#)
- Spera, M., Capuzzo Dolcetta, R., & Punzo, D. 2012, HiGPUs: Hermite's N-body Integrator Running on Graphic Processing Units, Astrophysics Source Code Library, ascl:[1207.002](#)
- Spinnato, P. F., Fellhauer, M., & Portegies Zwart, S. F. 2003, [MNRAS](#), **344**, [22](#)
- Spitzer, L. 1969, [ApJL](#), **158**, [L139](#)
- Spitzer, L. 1987, Dynamical Evolution of Globular Clusters (Princeton, NJ: Princeton Univ. Press)
- Springel, V. 2000, GADGET-2: A Code for Cosmological Simulations of Structure Formation, Astrophysics Source Code Library, ascl:[0003.001](#)
- Springel, V. 2005, [MNRAS](#), **364**, [1105](#)
- Stock, K., Cai, M. X., Spurzem, R., Kouwenhoven, M. B. N., & Portegies Zwart, S. 2020, [MNRAS](#), **497**, [1807](#)
- Takahashi, K., & Portegies Zwart, S. F. 1998, [ApJL](#), **503**, [L49](#)
- Taylor, B. 1715, Methodus Incrementorum Directa et Inversa (London: Typis Pearsonianis)
- Tremaine, S. 2015, [ApJ](#), **807**, [157](#)
- Trenti, M., & van der Marel, R. 2013, [MNRAS](#), **435**, [3272](#)
- van Elteren, A., Portegies Zwart, S., Pelupessy, I., Cai, M. X., & McMillan, S. L. W. 2019, [A&A](#), **624**, [A120](#)
- Verlet, L. 1967, [PhRv](#), **159**, [98](#)
- Verlinde, E. 2017, [ScPP](#), **2**, [016](#)
- von Zeipel, H. 1910, [AN](#), **183**, [345](#)
- Wang, L., Spurzem, R., Aarseth, S., et al. 2015, [MNRAS](#), **450**, [4070](#)
- Wang, L., Spurzem, R., Aarseth, S., et al. 2016, [MNRAS](#), **458**, [1450](#)
- Whitehead, A. J., McMillan, S. L. W., Vesperini, E., & Portegies Zwart, S. 2013, [ApJ](#), **778**, [118](#)
- Wisdom, J., & Holman, M. 1991, [AJ](#), **102**, [1528](#)
- Yoshida, H. 1993, [CeMDA](#), **56**, [27](#)
- Zemp, M. 2014, Halogen: Multimass Spherical Structure Models for N-Body Simulations, Astrophysics Source Code Library, ascl:[1407.020](#)
- Zemp, M., Moore, B., Stadel, J., Carollo, C. M., & Madau, P. 2008, [MNRAS](#), **386**, [1543](#)